

# Output-Based Error Estimation and Mesh Adaptation in Computational Fluid Dynamics

K.J. Fidkowski

Aero 623  
*University of Michigan*

Winter 2026

# Introduction

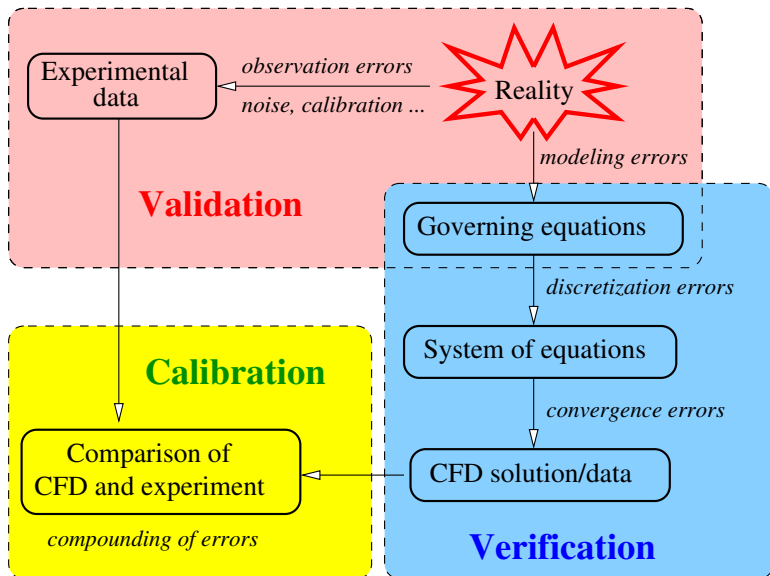
## **Complex CFD simulations are made possible by**

- Increasing computational power
- Improvements in numerical algorithms

## **New liability: ensuring accuracy of computations**

- Management by expert practitioners is not feasible for increasingly-complex flow fields
- Reliance on best-practice guidelines is an open-loop solution: numerical error is unchecked for novel configurations
- Output calculations are not yet sufficiently robust, even on relatively standard simulations

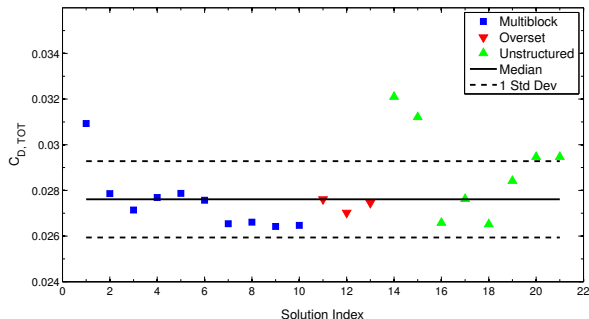
# Errors in simulations come from various sources



# Verification is important

## AIAA Drag Prediction Workshop III (2006)

- Wing-body geometry,  $M = 0.75$ ,  $C_L = 0.5$ ,  $Re = 5 \times 10^6$
- Drag computed with various state of the art CFD codes



## Differences in:

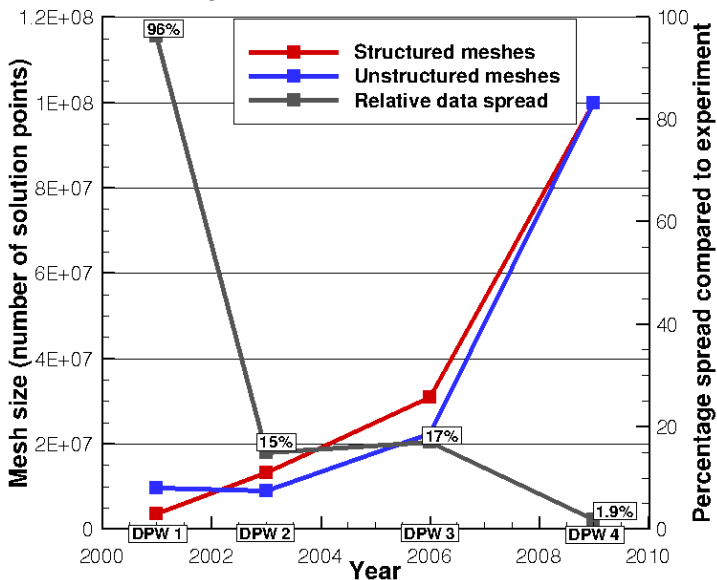
- Physical models
- Discretization
- **Mesh size distribution**

1 drag count ( $.0001 C_D$ )  $\approx$  4-8 passengers for a large aircraft



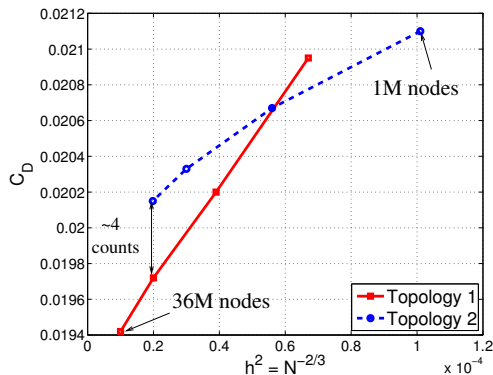
# Numerical errors have come down, at a large cost

Summary of AIAA DPW results (Ceze 2013)



# Uniform grid refinement may be misleading

- DPW III wing-alone case:  $M_\infty = 0.76$ ,  $Re = 5 \times 10^6$
- Two mesh sequences generated using best-practice mesh-generation guidelines
- Run on same code (turbulence model, solver, etc)



DPW III wing-alone  
case:  $M_\infty = 0.76$ ,  $Re = 5 \times 10^6$  [Mavriplis, 2007]

# Verification = control of numerical error

- Dominant source is **discretization error** (i.e. lack of appropriate mesh resolution)
- Controlling error means answering
  - ① How much error is present? (**error estimation**)
  - ② How can this error be reduced? (**mesh adaptation**)
- Possible strategies:

|                           | Error estimation? | Effective adaptation? |
|---------------------------|-------------------|-----------------------|
| Resource exhaustion       | No                | No                    |
| Expert assessment         | Maybe             | Maybe                 |
| Convergence studies       | Yes               | No                    |
| Comparison to experiments | Yes               | No                    |
| Feature-based adaptation  | No                | Maybe                 |
| Output-based methods      | Yes               | Yes                   |

# Improving CFD robustness

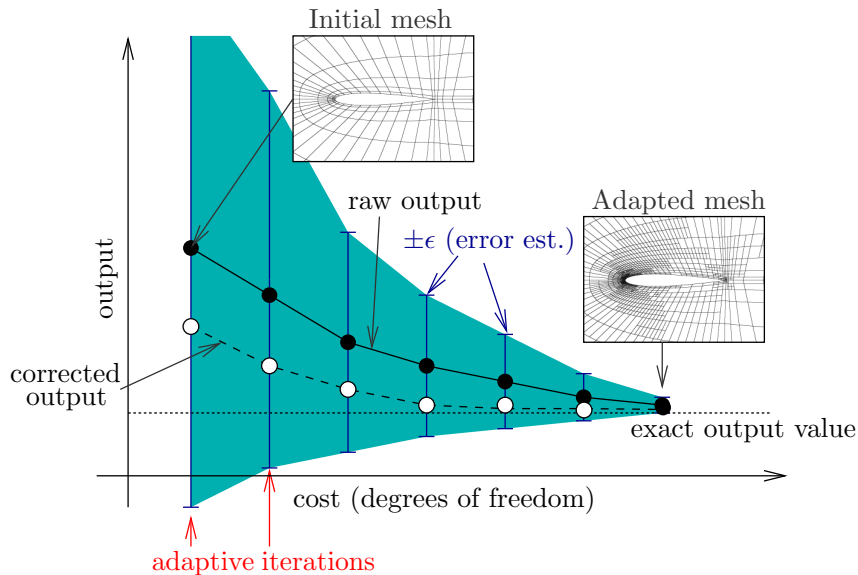
## Error estimation

- Error estimates on outputs of interest are necessary for confidence in CFD results
- Mathematical theory exists for obtaining such estimates
- Recent works demonstrate the success of this theory for aerospace applications

## Mesh adaptation

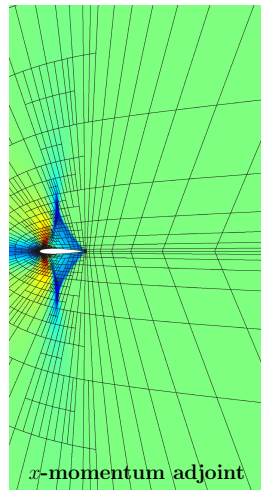
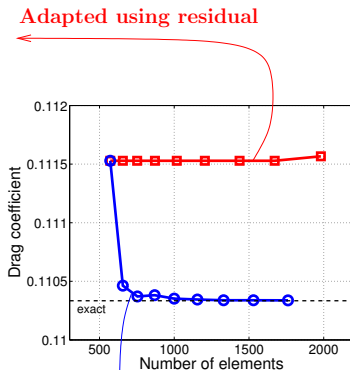
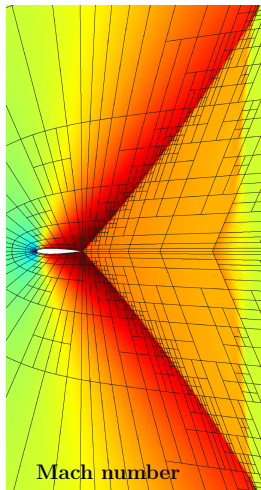
- Error estimation alone is not enough
- Engineering accuracy for complex aerospace simulations demands mesh adaptation to control numerical error
- Automated adaptation improves robustness by closing the loop in CFD analysis

# A typical output-adaptive result



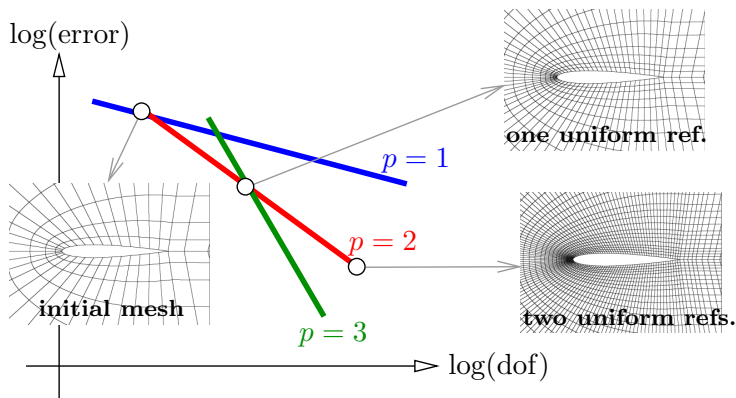
# Why not just adapt “obvious” regions?

Fishtail shock in  $M_\infty = 0.95$  inviscid flow over a NACA 0012 airfoil



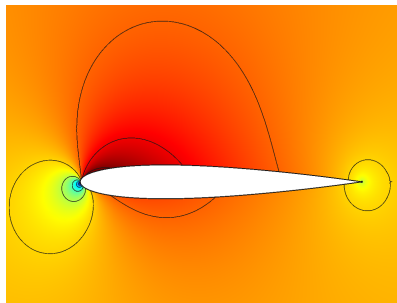
# The role of high order

- High-order methods: errors converge faster than 2<sup>nd</sup>-order
- Typically choose high-order methods for “smooth” problems, where we expect to see convergence plots that look like:

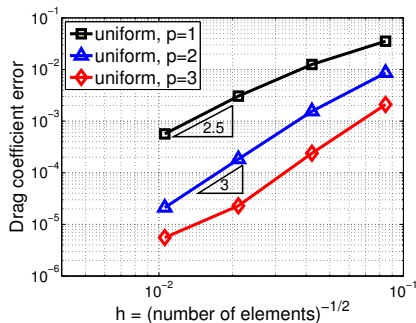


# Can aero applications benefit from high order?

- Question considered by recent high-order CFD workshops
- Aerospace applications usually have both smooth and singular features (shocks, trailing edges)
- Singularities can limit observed rates



Mach numbers (0–0.7), Euler flow over a NACA 0012 airfoil

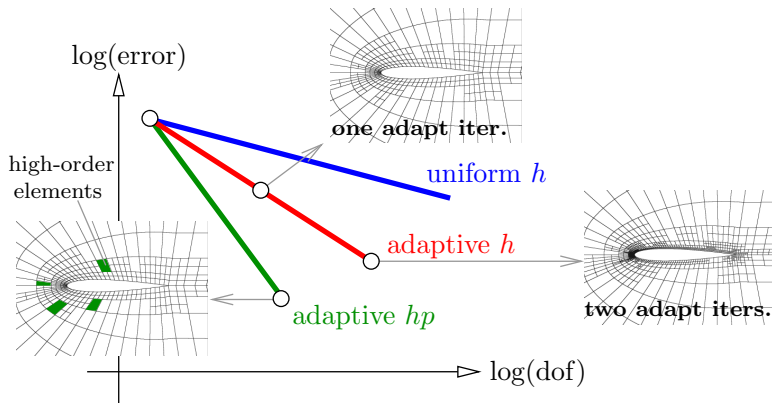


Drag error convergence



# High-order in mesh adaptation

- Adaptation can isolate singularities with small elements
- In many high-order methods, local  $p$ -enrichment is possible
- High-order just becomes another refinement tool for efficiently improving accuracy



# Conservation equations

$$\mathbf{r}(\mathbf{u}) \equiv \frac{\partial \mathbf{u}}{\partial t} + \nabla \cdot \vec{\mathbf{H}}(\mathbf{u}, \nabla \mathbf{u}) = \mathbf{0}$$

- $\mathbf{u} \in \mathbb{R}^s$  is the state vector
- $\vec{\mathbf{H}} \in [\mathbb{R}^s]^d$  is the total flux with spatial components

$$\mathbf{H}_i = \underbrace{\mathbf{F}_i(\mathbf{u})}_{\text{inviscid flux}} + \underbrace{\mathbf{G}_i(\mathbf{u}, \nabla \mathbf{u})}_{\text{viscous flux}}$$

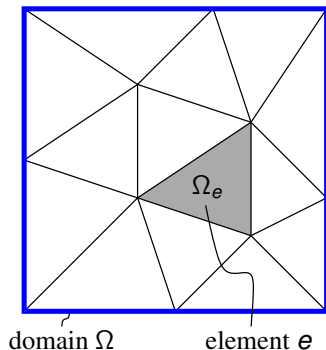
- $1 \leq i \leq d = \text{spatial dimension}$
- The viscous flux is linear in the state gradient

$$\mathbf{G}_i(\mathbf{u}, \nabla \mathbf{u}) = -\mathbf{K}_{ij}(\mathbf{u}) \partial_j \mathbf{u}$$

# Finite-element solution approximation

Polynomials of order  $p_e$  on each element:

$$\mathbf{u}_h(\vec{x}) \approx \sum_{e=1}^{N_e} \sum_{n=1}^{N_{p_e}} \mathbf{u}_{en} \phi_{en}(\vec{x})$$



$N_e$  = # of elements

$N_{p_e}$  = # of basis fcn's on element  $e$

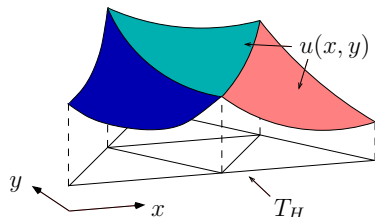
$\phi_{en}(\vec{x})$  =  $n^{\text{th}}$  basis fcn of order  $p_e$  on  $e$

$p_e$  = approximation order on element  $e$

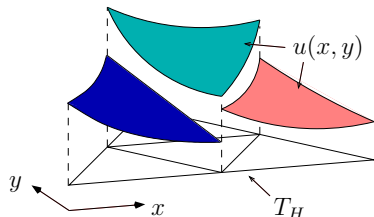
$\mathbf{u}_{en}$  = vector of  $s$  coefficients on  $n^{\text{th}}$  basis function on element  $e$

# Discontinuous basis functions

## Continuous Galerkin (CG)



## Discontinuous Galerkin (DG)



- DG approximation space: no inter-element continuity,

$$\mathbf{u} \in \mathcal{V}_h = [\mathcal{V}_h]^s, \quad \mathcal{V}_h = \left\{ u \in L^2(\Omega) : u|_{\Omega_e} \in \mathcal{P}^{p_e}(\Omega_e) \quad \forall \Omega_e \in T_h \right\}$$

- Equations: multiply by test functions and integrate by parts
- Elements coupled together through upwind flux functions
- $p = 0 \Rightarrow$  first-order finite-volume method

# Weak form

- Multiply PDE by test functions  $\mathbf{v}_h \in \mathcal{V}_h$  and integrate over  $\Omega$ :

$$\mathcal{R}_h(\mathbf{u}_h, \mathbf{v}_h) \equiv \int_{\Omega} \mathbf{v}_h^T \mathbf{r}(\mathbf{u}_h) d\Omega = 0, \quad \forall \mathbf{v}_h \in \mathcal{V}_h$$

- Integrate by parts over each element,

$$\begin{aligned} \mathcal{R}_h(\mathbf{u}_h, \mathbf{v}_h|_{\Omega_e}) &= \int_{\Omega_e} \mathbf{v}_h^T \partial_t \mathbf{u}_h d\Omega - \int_{\Omega_e} \partial_i \mathbf{v}_h^T \mathbf{H}_i d\Omega \\ &\quad + \underbrace{\int_{\partial\Omega_e} \mathbf{v}_h^{+T} \hat{\mathbf{H}} ds}_{\text{interface/boundary flux}} \end{aligned}$$

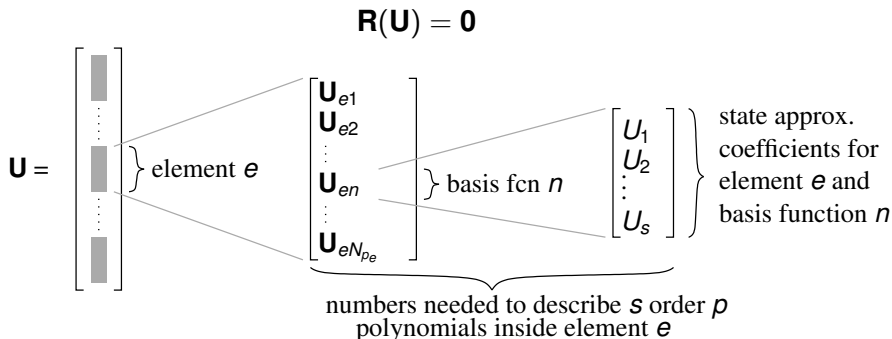
- $()^+$  denotes an interface quantity taken from element interior
- $\hat{\mathbf{H}}$  is a single-valued numerical flux function (as in FV)
- Additional stabilization and adjoint-consistency terms appear for viscous discretizations

# Discrete system

- Discrete residual on element  $e$  for  $n^{\text{th}}$  test function,

$$\mathbf{R}_{en} \equiv \{\mathcal{R}_h(\mathbf{u}_h, \phi_{en} \mathbf{e}_r)\}_{r=1\dots s} \in \mathbb{R}^s$$

- We lump all residuals and states into single vectors (size  $N$ ),



# Local sensitivities

- Suppose  $N_\mu$  parameters affect our PDE, but we only have one scalar output,  $J(\mathbf{U})$ :

$$\underbrace{\mu}_{\text{inputs} \in \mathbb{R}^{N_\mu}} \rightarrow \underbrace{\mathbf{R}(\mathbf{U}, \mu) = \mathbf{0}}_{N \text{ equations}} \rightarrow \underbrace{\mathbf{U}}_{\text{state} \in \mathbb{R}^N} \rightarrow \underbrace{J(\mathbf{U})}_{\text{output (scalar)}}$$

- We are interested in how  $J$  changes with  $\mu$ ,

$$\frac{dJ}{d\mu} \in \mathbb{R}^{1 \times N_\mu} = N_\mu \text{ sensitivities}$$

- Brute force approach: perturb each entry in  $\mu$  individually, re-solve the PDE, and measure the perturbation in the output

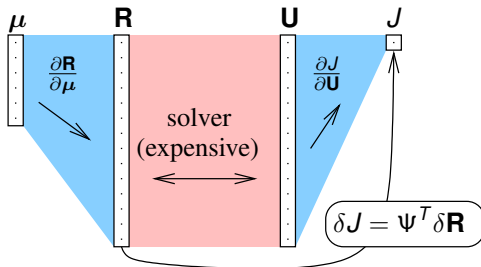
This is inefficient for large  $N_\mu$

# The discrete adjoint

- We can efficiently compute sensitivities using a discrete adjoint vector,  $\psi \in \mathbb{R}^N$ ,

$$\frac{dJ}{d\mu} = \psi^T \frac{\partial \mathbf{R}}{\partial \mu}$$

- Each entry in  $\psi$  is the sensitivity of  $J$  to residual source perturbations in the corresponding entry in  $\mathbf{R}$





# The discrete adjoint equation

- Consider a small perturbation  $\delta \mathbf{R}$  to the residual
- The resulting (linearized) state perturbation,  $\delta \mathbf{U}$  satisfies

$$\frac{\partial \mathbf{R}}{\partial \mathbf{U}} \delta \mathbf{U} + \delta \mathbf{R} = 0$$

- Also linearizing the output we have,

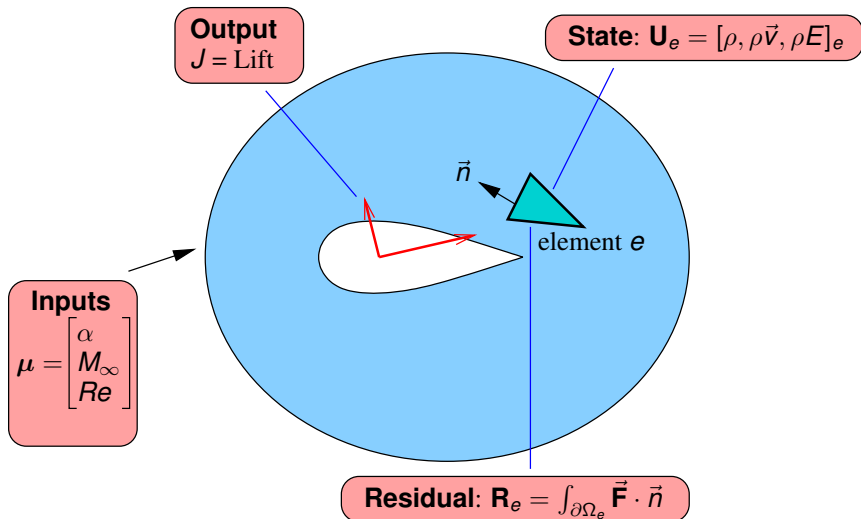
$$\delta J = \underbrace{\frac{\partial J}{\partial \mathbf{U}} \delta \mathbf{U}}_{\text{adjoint definition}} = \underbrace{\psi^T \delta \mathbf{R}}_{\text{linearized equations}} = -\psi^T \frac{\partial \mathbf{R}}{\partial \mathbf{U}} \delta \mathbf{U}$$

- Requiring the above to hold for arbitrary perturbations yields the linear *discrete adjoint equation*

$$\left( \frac{\partial \mathbf{R}}{\partial \mathbf{U}} \right)^T \psi + \left( \frac{\partial J}{\partial \mathbf{U}} \right)^T = \mathbf{0}$$

# Adjoint in aerodynamics

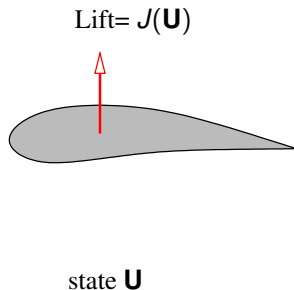
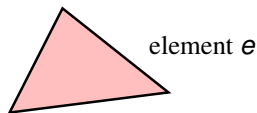
*Consider flow over an airfoil:*



# Output sensitivity to residuals: the adjoint

The lift adjoint  $\Psi$  is the sensitivity of lift to residual sources.

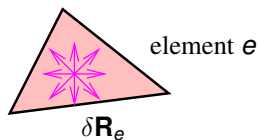
We have a solution  $\mathbf{U}$  when  $\mathbf{R} = 0$



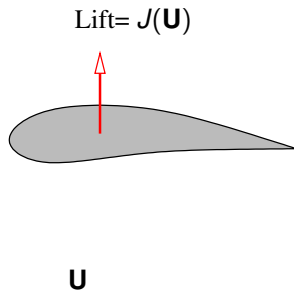
# Output sensitivity to residuals: the adjoint

The lift adjoint  $\Psi$  is the sensitivity of lift to residual sources.

We have a solution  $\mathbf{U}$  when  $\mathbf{R} = 0$



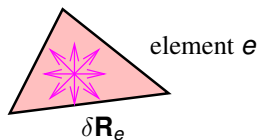
What if we add a residual source,  $\delta \mathbf{R}_e$ ?



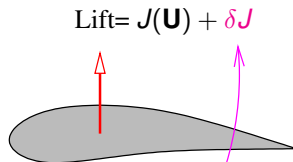
# Output sensitivity to residuals: the adjoint

The lift adjoint  $\Psi$  is the sensitivity of lift to residual sources.

We have a solution  $\mathbf{U}$  when  $\mathbf{R} = 0$



What if we add a residual source,  $\delta \mathbf{R}_e$ ?

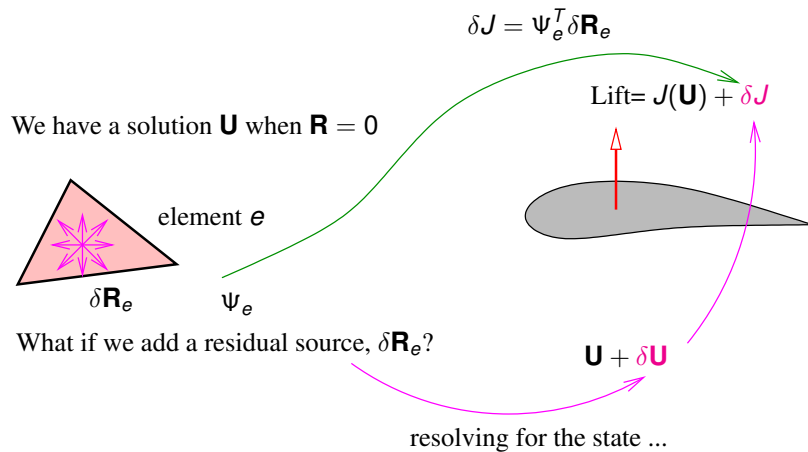


$\mathbf{U} + \delta \mathbf{U}$

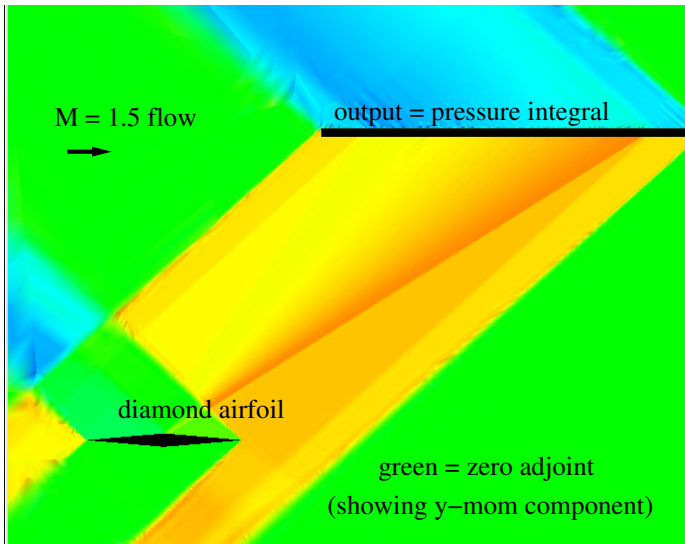
resolving for the state ...

# Output sensitivity to residuals: the adjoint

The lift adjoint  $\psi$  is the sensitivity of lift to residual sources.

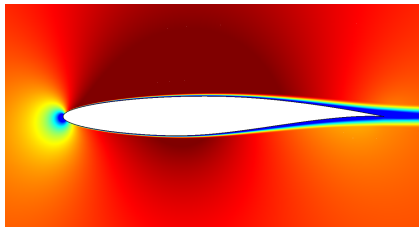


# Sample steady adjoint solution

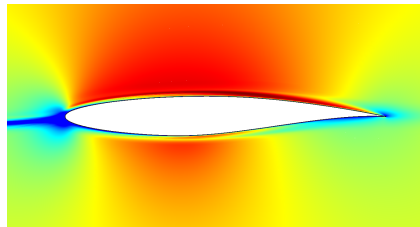


# Another steady adjoint solution

RAE 2822,  $M_\infty = 0.5$ ,  $Re = 10^5$ ,  $\alpha = 1^\circ$



x-momentum primal state



cons. of x-mom drag adjoint

- Adjoint shares similar qualitative features with primal
- Note wake “reversal” in adjoint solution
- The discrete adjoint solution approximates the continuous adjoint when the discretization is *adjoint consistent*

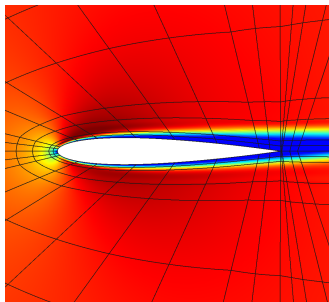


# Adjoint verification

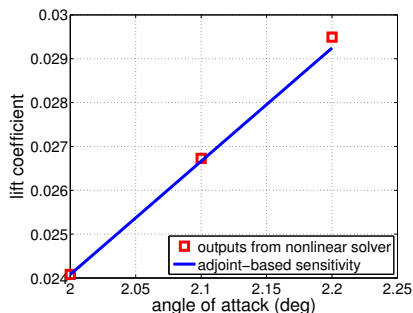
- We can verify the discrete adjoint with a sensitivity analysis,

$$\frac{dJ}{d\alpha} = \psi^T \frac{\partial \mathbf{R}}{\partial \alpha} + \frac{\partial J}{\partial \alpha}$$

- Compare to finite-difference sensitivity calculation
- Example: NACA 0012 airfoil in  $Re = 5000$  flow



Mach number contours



Lift coefficient sensitivity

# Adjoint implementation

- The discrete adjoint,  $\Psi$ , is obtained by solving a linear system
- This system involves linearizations about the primal solution,  $\mathbf{U}$ , which is generally obtained first
- When the full Jacobian matrix,  $\frac{\partial \mathbf{R}}{\partial \mathbf{U}}$ , and an associated linear solver are available, the transpose linear solution is straightforward
- When the Jacobian matrix is not stored, the discrete adjoint solution is more involved: all operations in the primal solver must be linearized, transposed, and applied in reverse order
- In unsteady discretizations, the adjoint must be marched backward in time from the final to the initial state

# Output error estimation

**We want:**  $\delta J = J_H(\mathbf{U}_H) - J(\mathbf{U})$

This is the difference between  $J$  computed with the discrete system solution,  $\mathbf{U}_H$ , and  $J$  computed with the *exact* solution,  $\mathbf{U}$

**We'll settle for:**  $\delta J = J_H(\mathbf{U}_H) - J_h(\mathbf{U}_h)$

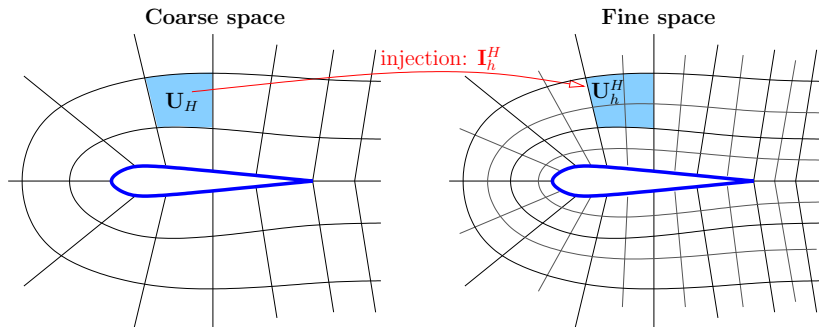
This is the difference in  $J$  relative to a finer discretization ( $h$ )

coarse space:  $\rightarrow \underbrace{\mathbf{R}_H(\mathbf{U}_H) = 0}_{N_H \text{ equations}} \rightarrow \underbrace{\mathbf{U}_H}_{\text{state} \in \mathbb{R}^{N_H}} \rightarrow \underbrace{J_H(\mathbf{U}_H)}_{\text{output (scalar)}}$

fine space:  $\rightarrow \underbrace{\mathbf{R}_h(\mathbf{U}_h) = 0}_{N_h \text{ equations}} \rightarrow \underbrace{\mathbf{U}_h}_{\text{state} \in \mathbb{R}^{N_h}} \rightarrow \underbrace{J_h(\mathbf{U}_h)}_{\text{output (scalar)}}$

# Fine-space injection

- The fine space can arise from  $h$  or  $p$  refinement
- Define an injection of the coarse state into the fine space



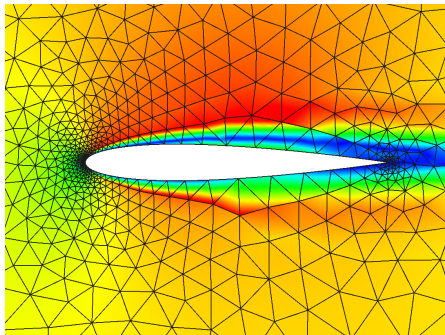
- $U_h^H$  will generally not satisfy the fine-space equations,

$$\mathbf{R}_h(\mathbf{U}_h^H) \neq \mathbf{0}$$

# Fine-space residuals

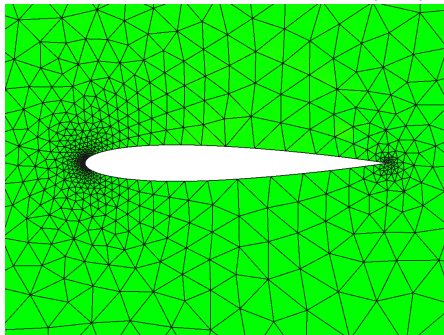
- A finer space (e.g. order enrichment) can uncover residuals in a converged solution
- Example: NACA 0012 at  $\alpha = 2^\circ$  in  $Re = 5000$ ,  $M_\infty = 0.5$  flow

Coarse space state,  $\mathbf{U}_H$



$$\rho_H = 1$$

Coarse space residual,  $\mathbf{R}_H(\mathbf{U}_H)$

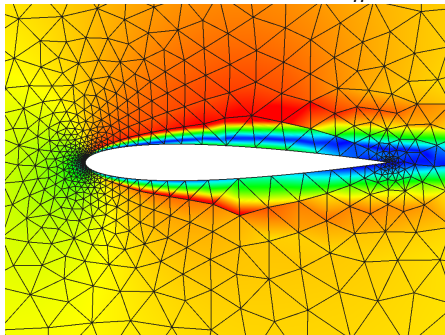


Zero as expected

# Fine-space residuals

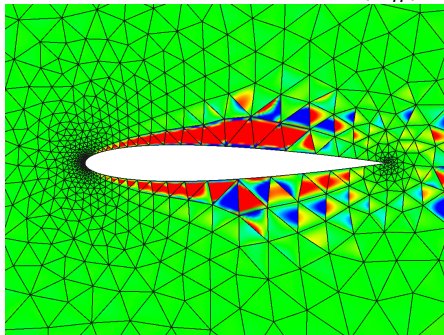
- A finer space (e.g. order enrichment) can uncover residuals in a converged solution
- Example: NACA 0012 at  $\alpha = 2^\circ$  in  $Re = 5000$ ,  $M_\infty = 0.5$  flow

Injected state,  $\mathbf{U}_h^H$



$p_h = 2$

Fine space residual,  $\mathbf{R}_h(\mathbf{U}_h^H)$



Nonzero: new info

# The adjoint-weighted residual

- $\mathbf{U}_h^H$  solves a *perturbed* fine-space problem

$$\text{find } \mathbf{U}'_h \text{ such that: } \mathbf{R}_h(\mathbf{U}'_h) - \underbrace{\mathbf{R}_h(\mathbf{U}_h^H)}_{\delta \mathbf{R}_h} = 0 \quad \Rightarrow \text{answer: } \mathbf{U}'_h = \mathbf{U}_h^H$$

- The fine-space adjoint,  $\Psi_h$ , then tells us to expect an output perturbation of

$$\underbrace{J_h(\mathbf{U}_h^H) - J_h(\mathbf{U}_h)}_{\approx \delta J} = \Psi_h^T \delta \mathbf{R}_h = -\Psi_h^T \mathbf{R}_h(\mathbf{U}_h^H)$$

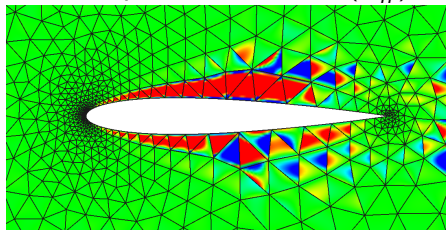
Becker and Rannacher, 2007   Giles and Pierce, 1997

- This equation assumes small perturbations (e.g. if nonlinear)
- In summary, we have an *adjoint-weighted residual*:

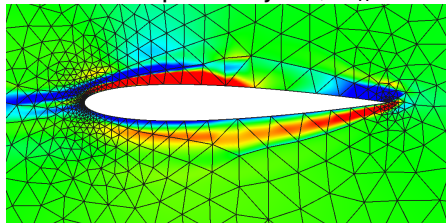
$$\delta J \approx -\Psi_h^T \mathbf{R}_h(\mathbf{U}_h^H)$$

# Adjoint-weighted residual example

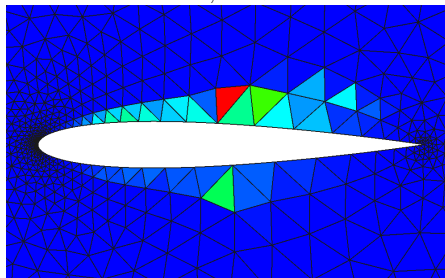
Fine space residual,  $\mathbf{R}_h(\mathbf{U}_h^H)$



Fine space adjoint,  $\psi_h$



Error indicator,  
 $\epsilon_e = |\psi_{h,e}^T \mathbf{R}_{h,e}(\mathbf{U}_h^H)|$



$$\text{Output error: } \delta J \approx -\psi_h^T \mathbf{R}_h(\mathbf{U}_h^H)$$

*Idea: adapt where  $\epsilon_e$  is high, to  
reduce the residual there*



# Two more definitions

## Corrected output

$$J_H^{\text{corrected}} = J_H - \delta J$$

- Should converge faster than  $J_H$
- Remaining error = error left in corrected output

## Error effectivity

$$\eta_H = \frac{J_H(\mathbf{U}_H) - J_h(\mathbf{U}_h)}{J_H(\mathbf{U}_H) - J}$$

- $J$  = exact output
- We want  $\eta_H$  close to 1
- Effectivity is affected by choice of fine space

# Error convergence tests

- Expect

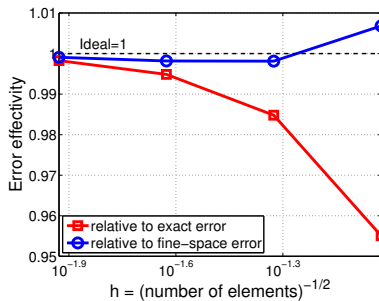
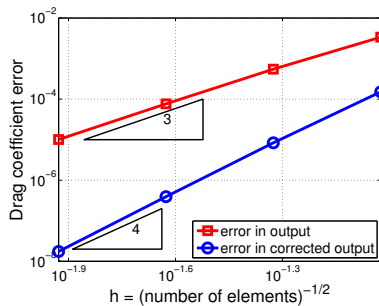
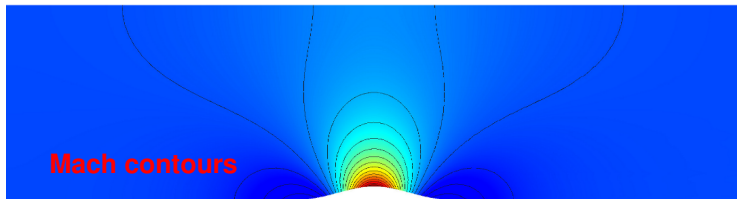
$$\text{error} = Ch^k, \quad \text{as } h \rightarrow 0$$

- $k$  = rate of convergence
- $h$  = measure of element size (relative value important)
- For 2D uniform-refinement studies, can use  $h = \sqrt{1/N_e}$
- Taking a log of the above equation,

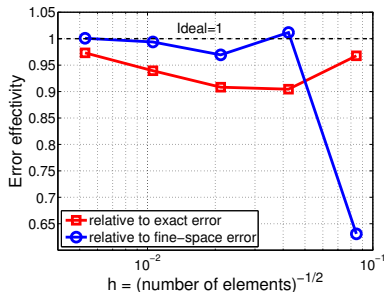
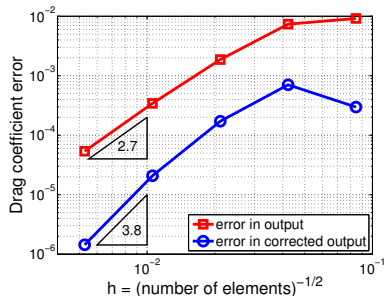
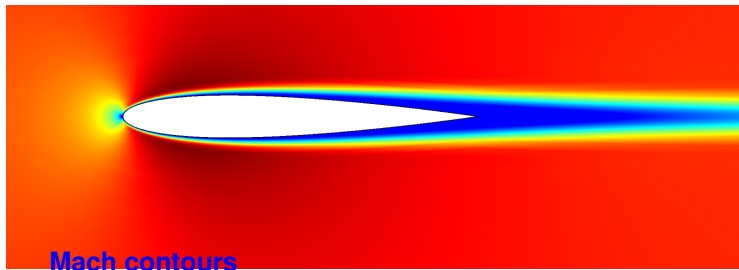
$$\log(\text{error}) = \log C + k \log \left( \sqrt{\frac{1}{N_e}} \right)$$

- We can measure  $k$  by plotting  $\log(\text{error})$  versus  $\log(h)$

# Drag error in inviscid flow over a bump



# Drag error in viscous flow over an airfoil



# Approximations

How do we calculate  $\Psi_h$  = the adjoint on the fine space?

## Options:

- ① Solve for  $\mathbf{U}_h$  and then  $\Psi_h$  – expensive! Potentially still useful to drive adaptation. Solín and Demkowicz, 2004
- ② Solve for the coarse space adjoint,  $\Psi_H$ , and:
  - ① Reconstruct  $\Psi_H$  on the fine space using a higher-accuracy stencil. Smoothness assumption on adjoint.  
Rannacher, 2001 Barth and Larson, 2002 Venditti and Darmofal, 2002 Lu, 2005 Fidkowski and Darmofal, 2007
  - ② Initialize  $\Psi_h$  with  $\Psi_H$  and take a few iterative solution (smoothing) steps on the fine space.  
Barter and Darmofal, 2008 Oliver and Darmofal, 2008

# Corrections and remainders

- Define  $\Psi_h^H \equiv \mathbf{I}_h^H \Psi_H =$  injection of  $\Psi_H$  into  $h$ .
- Define the adjoint perturbation,  $\delta\Psi_h \equiv \Psi_h^H - \Psi_h$
- Rewrite the adjoint-weighted residual as:

$$\delta J = \underbrace{-\left(\Psi_h^H\right)^T \mathbf{R}_h(\mathbf{U}_h^H)}_{\text{computable correction}} + \underbrace{\left(\delta\Psi_h\right)^T \mathbf{R}_h(\mathbf{U}_h^H)}_{\text{remaining error}} + \underbrace{\mathcal{O}(\delta\mathbf{U}_h, \delta\Psi_h)^2}_{\text{error in estimate}}$$

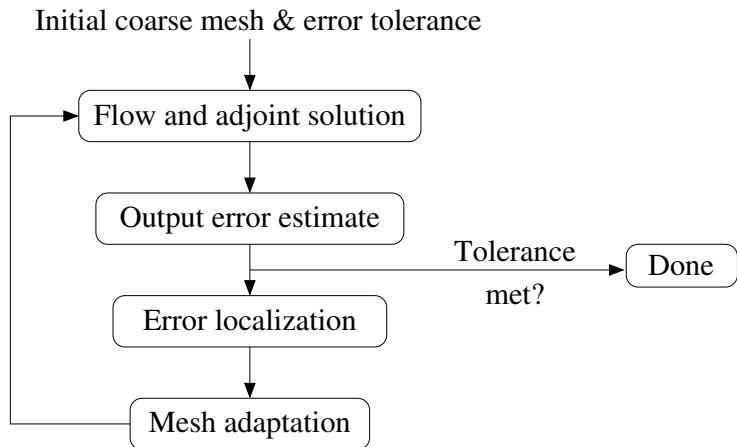
- The computable correction is tempting to use directly, but:
  - It does not incorporate fine-space information  $\Rightarrow$  it performs poorly as an adaptive indicator
  - It is zero for FEM with Galerkin orthogonality
- For nonlinear problems the “error in the estimate” can be reduced to third-order via [Rannacher, 2001](#):

$$\delta J \approx -\left(\Psi_h^H\right)^T \mathbf{R}_h(\mathbf{U}_h^H) + \frac{1}{2} (\delta\Psi_h)^T \mathbf{R}_h(\mathbf{U}_h^H) + \frac{1}{2} (\delta\mathbf{U}_h)^T \mathbf{R}_h^\psi(\Psi_h^H)$$

# Error estimation summary

- 1 Solve the coarse-discretization forward and adjoint problems:  $\mathbf{U}_H$  and  $\Psi_H$
- 2 Pick a fine discretization “ $h$ ” (mesh refinement or order enrichment)
- 3 Calculate or approximate  $\Psi_h =$  adjoint on the fine space
- 4 Project  $\mathbf{U}_H$  onto the fine discretization and calculate the residual  $\mathbf{R}_h(\mathbf{U}_h^H)$
- 5 Weight the fine-space residual with the fine-space adjoint to obtain the output error estimate
- 6 The computed output error  $\delta J$  is an estimate of the true error, not a bound

# Mesh adaptation





# Error localization

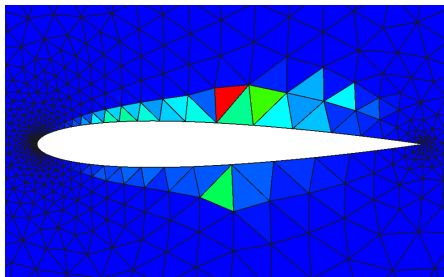
- Recall that the adjoint-weighted residual expression for the output error involves a sum over elements ( $e$ )

$$J_H(\mathbf{U}_H) - J_h(\mathbf{U}_h) \approx -\boldsymbol{\Psi}_h^T \mathbf{R}_h(\mathbf{U}_h^H) = -\sum_e \boldsymbol{\Psi}_{he}^T \mathbf{R}_{he}(\mathbf{U}_h^H)$$

- The absolute-value of each element's contribution to the error is the **error indicator** on that element

$$\epsilon_e \equiv \left| \boldsymbol{\Psi}_{he}^T \mathbf{R}_{he}(\mathbf{U}_h^H) \right|$$

Right : plot of error indicator for a  
viscous DG simulation,  $p_H = 1$ ,  
 $p_h = 2$



# Output-based mesh adaptation

## Motivating ideas

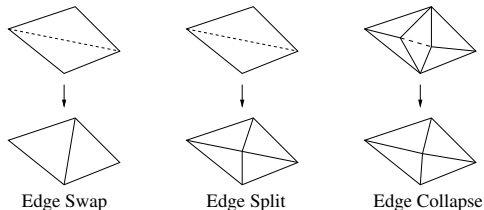
- The error indicator ( $\epsilon_e$ ) identifies elements with large adjoint-weighted residuals
- Locally refining a mesh reduces local residuals
- So we can reduce the output error by refining those elements that have a high  $\epsilon_e$

## Adaptation choices

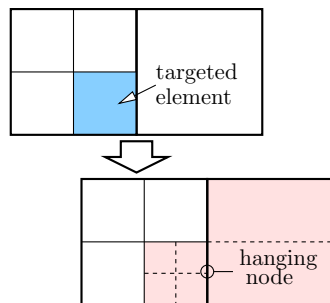
- Local refinement versus global re-meshing
- Which/how many elements should be targeted?
- Isotropic versus anisotropic refinement
- $h$ ,  $p$ , or  $hp$  mechanics
- Should coarsening be allowed?

# Local mesh modification

- Modify the mesh incrementally (mesh generation is hard)
- Often more robust than global re-meshing
- With node movement, can be flexible for unstructured meshes
- Hanging nodes easily supported in DG



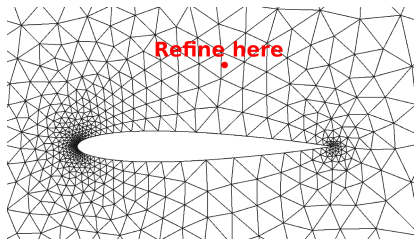
Unstructured local mesh operators



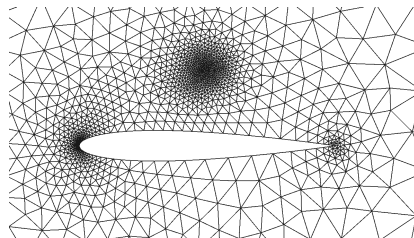
Hanging-node refinement

# Global re-meshing

- Entire mesh is re-generated
- Current mesh still plays a role in defining a *Riemannian metric*
- Useful software in 2D: Bi-dimensional Anisotropic Mesh Generator (BAMG)
- Example of refinement near a single point:



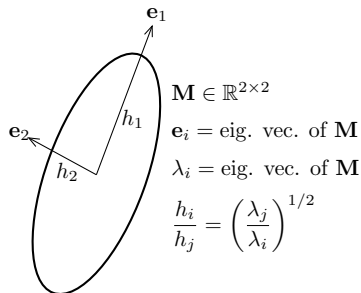
Original mesh



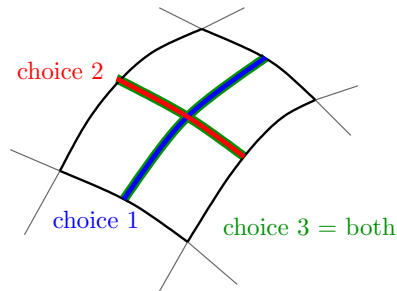
After refinement

# Incorporating anisotropy

- Crucial for high-Reynolds number simulations, esp. in 3D
- Can come in via a metric or discrete hanging-node “slices”



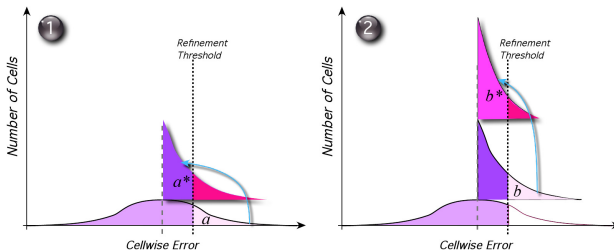
Anisotropic metric in 2D



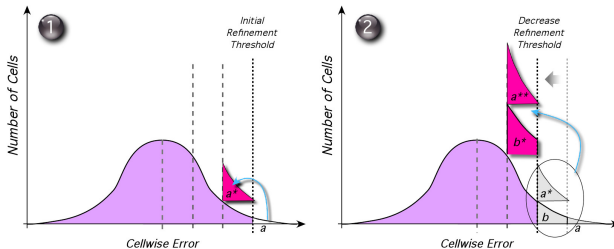
Hanging-node choices

- Assess need for anisotropy by
  - Looking at derivatives of a scalar quantity (Mach number)
  - Solving local sub-problems to determine impact of anisotropy directly on the output error

# Targeting strategies Nemec et al , 2008



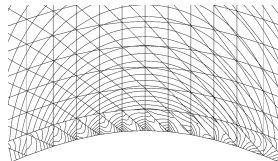
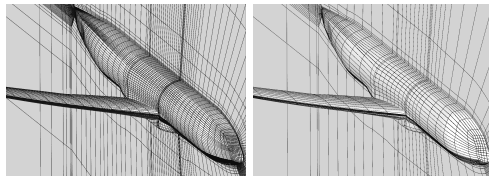
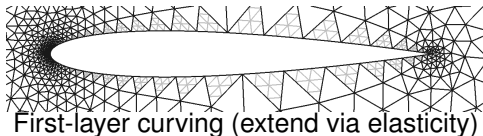
Constant threshold: refine all elements above a constant error indicator



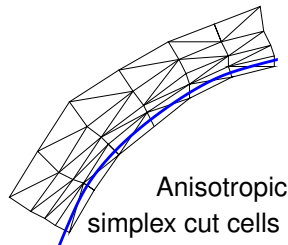
Decreasing threshold: threshold decreases with each iteration

# Curved boundaries

- DG needs an accurate representation of curved boundaries
- Curving elements is not easy
- Tangling is hard to avoid, especially in 3D anisotropic elements

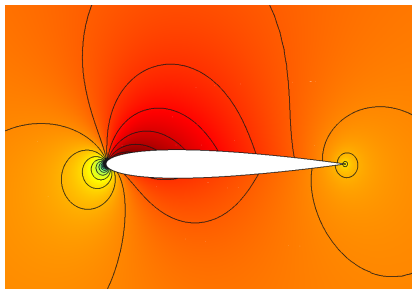


$p = 2$  Euler flow over a  
linear-element bump  
representation

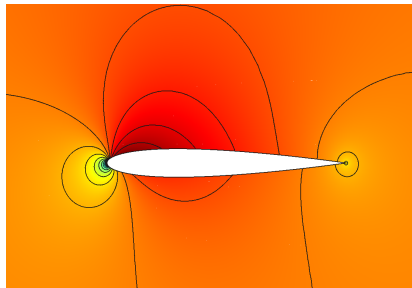


# Inviscid flow over an airfoil

NACA 0012, Euler,  $M_\infty = 0.5$ ,  $\alpha = 2^\circ$



Mach number contours



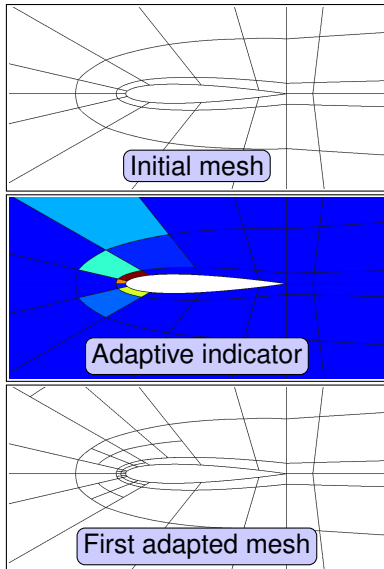
Drag adjoint ( $x$ -momentum)

- Output  $J = \text{drag}$  (expect  $\sim 0$ )
- Compare hanging-node adaptation to uniform refinement
- Look at approximation orders  $p = 1$  and  $p = 2$

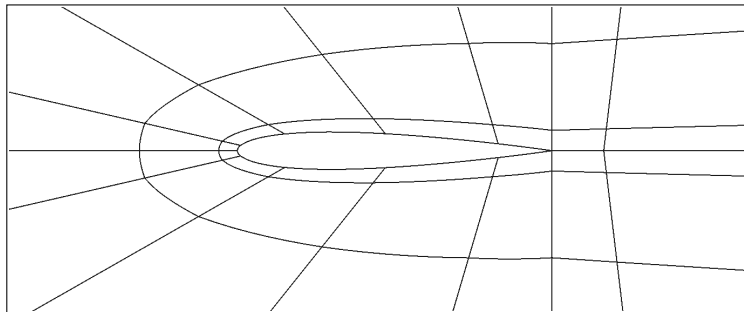
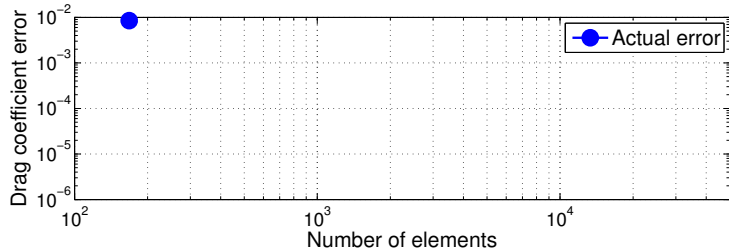


# Inviscid flow: hanging-node adaptation

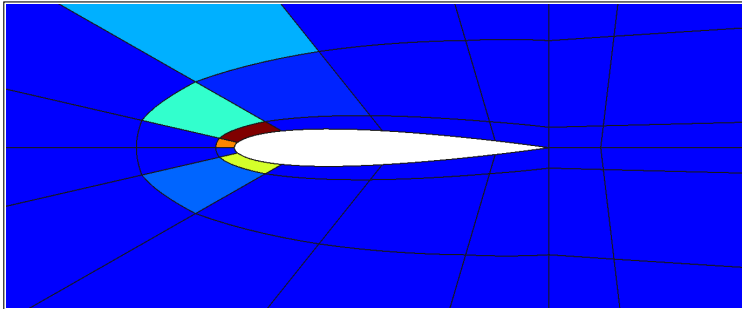
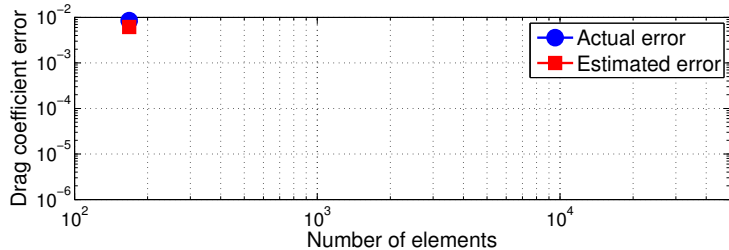
- Isotropic hanging-node refinement
- Fine space =  $p + 1$
- Fixed fraction  $f^{\text{adapt}} = 5\%$
- 20 adaptive iterations
- No coarsening
- Use adjoint-weighted residual  $\delta J$  as correction
- “Exact” output from a  $p = 3$  fine-mesh solver
- *Right:  $p = 2$  first adaptation*



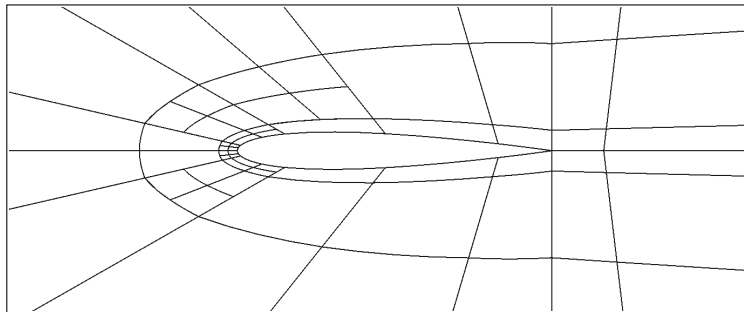
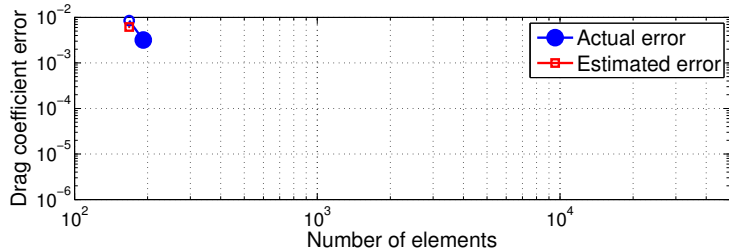
# Inviscid flow: $p = 2$ mesh sequence



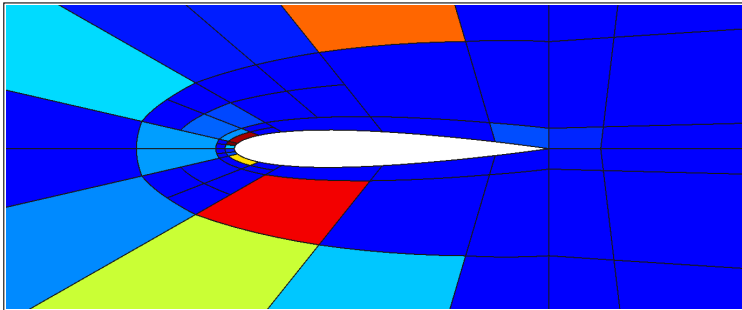
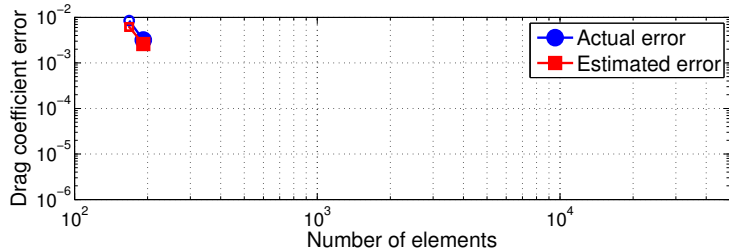
# Inviscid flow: $p = 2$ mesh sequence



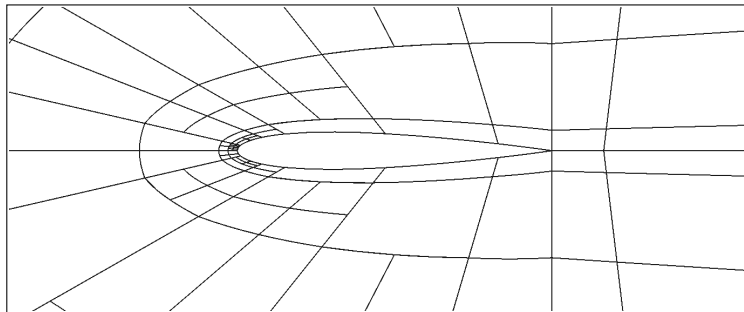
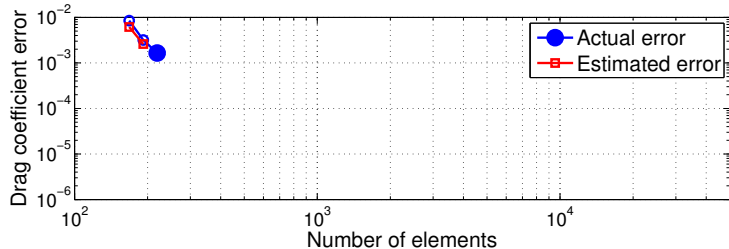
# Inviscid flow: $p = 2$ mesh sequence



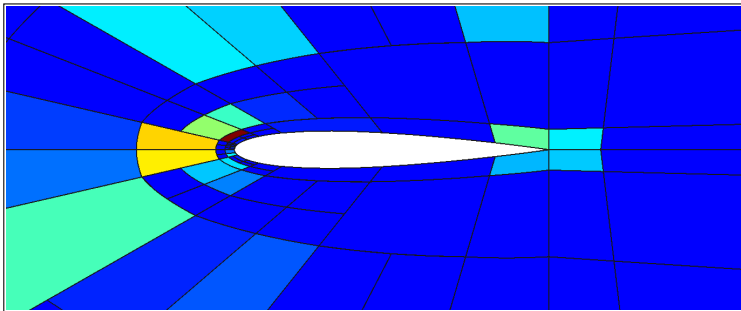
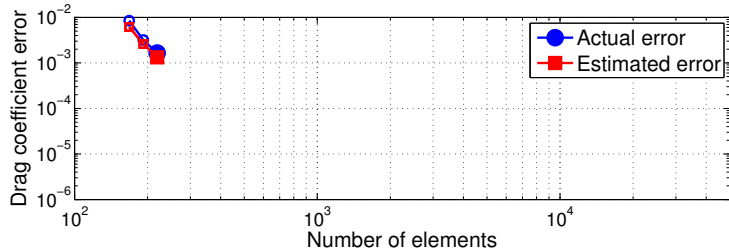
# Inviscid flow: $p = 2$ mesh sequence



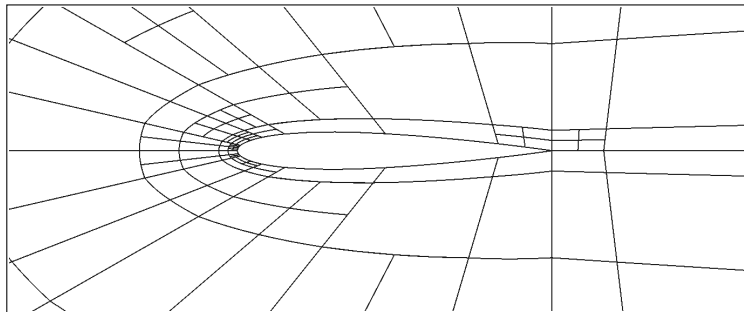
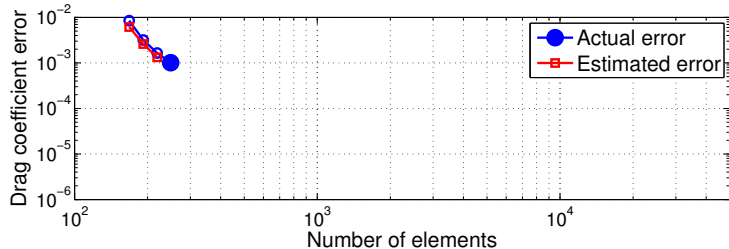
# Inviscid flow: $p = 2$ mesh sequence



# Inviscid flow: $p = 2$ mesh sequence

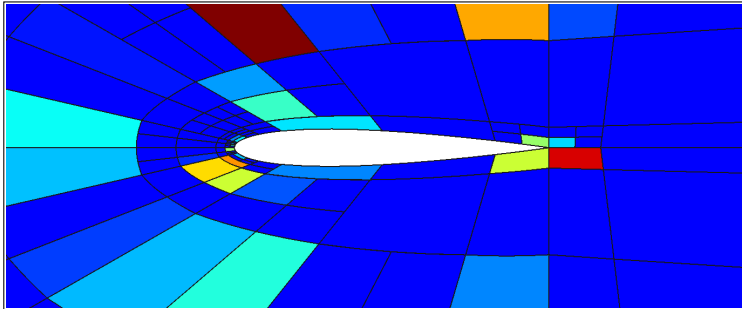
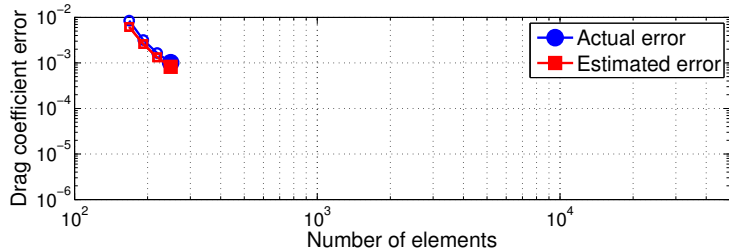


# Inviscid flow: $p = 2$ mesh sequence

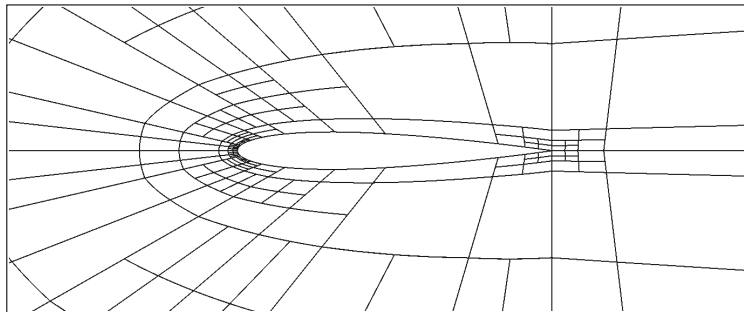
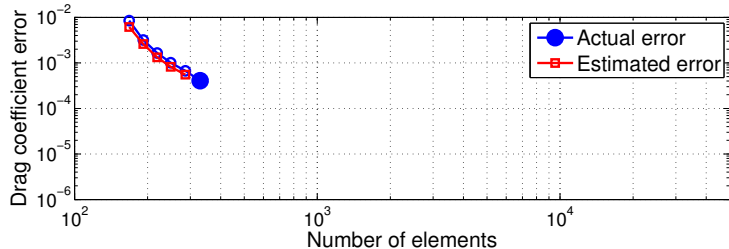




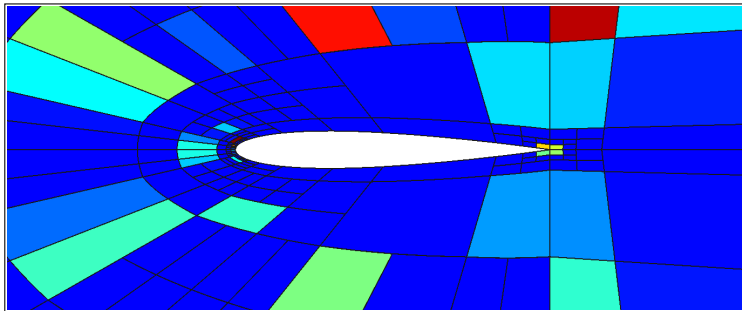
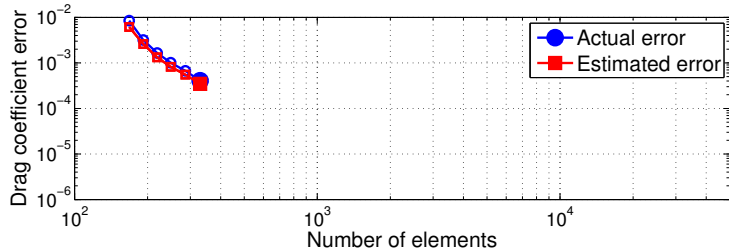
# Inviscid flow: $p = 2$ mesh sequence



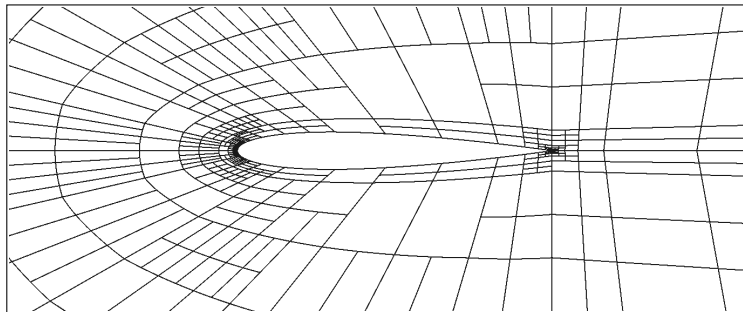
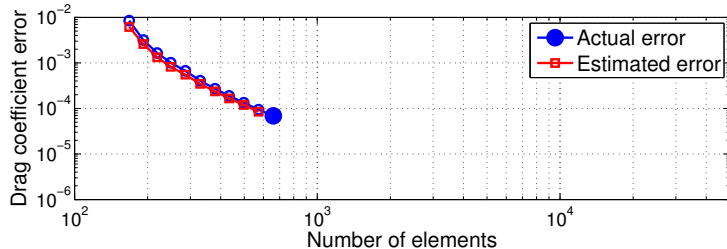
# Inviscid flow: $p = 2$ mesh sequence



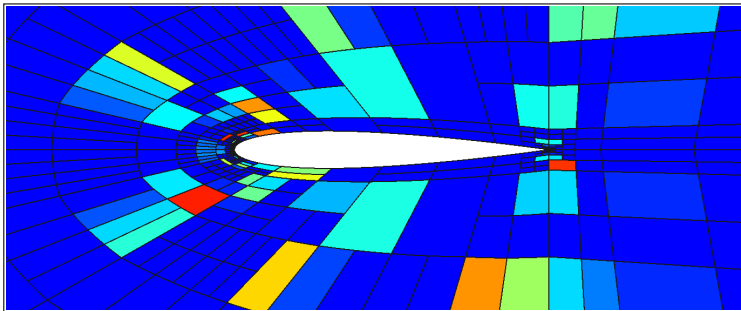
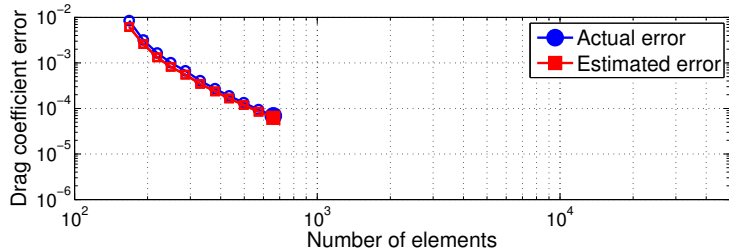
# Inviscid flow: $p = 2$ mesh sequence



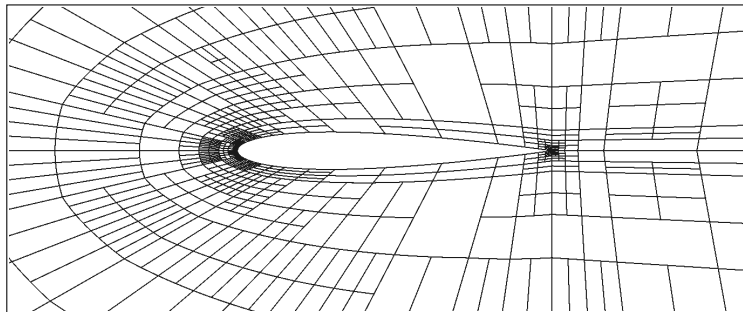
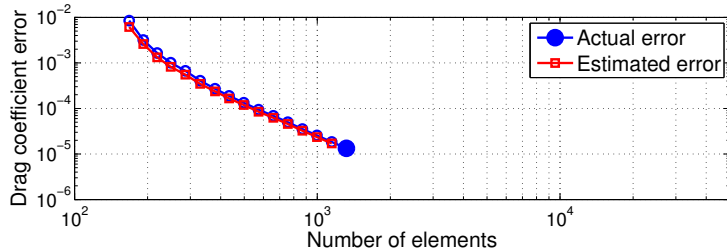
# Inviscid flow: $p = 2$ mesh sequence



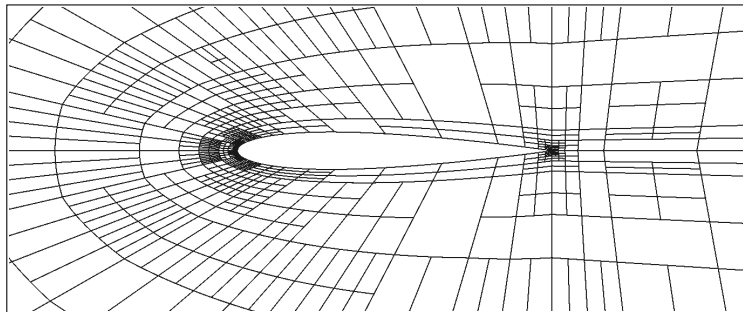
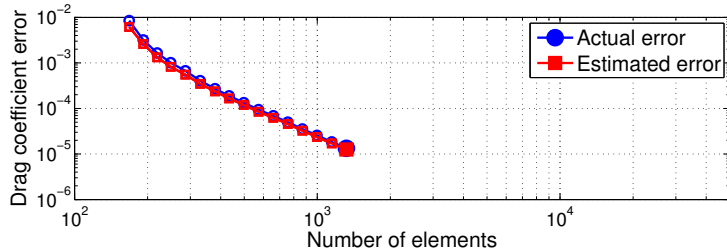
# Inviscid flow: $p = 2$ mesh sequence



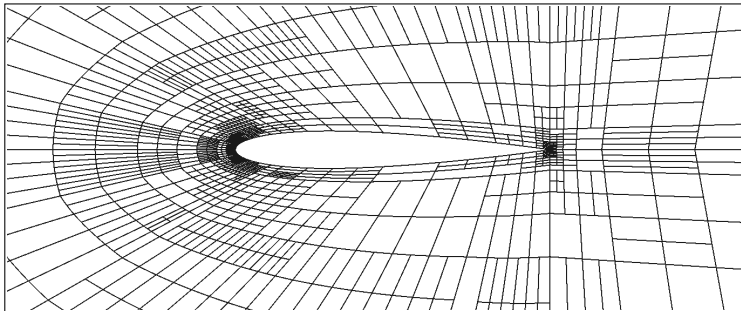
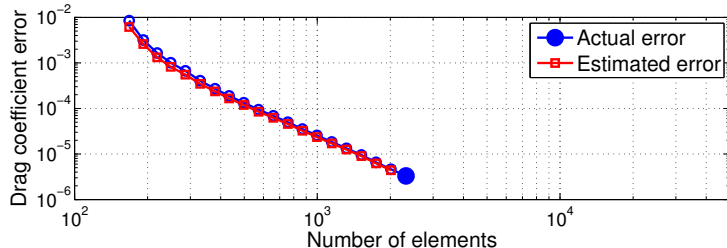
# Inviscid flow: $p = 2$ mesh sequence



# Inviscid flow: $p = 2$ mesh sequence

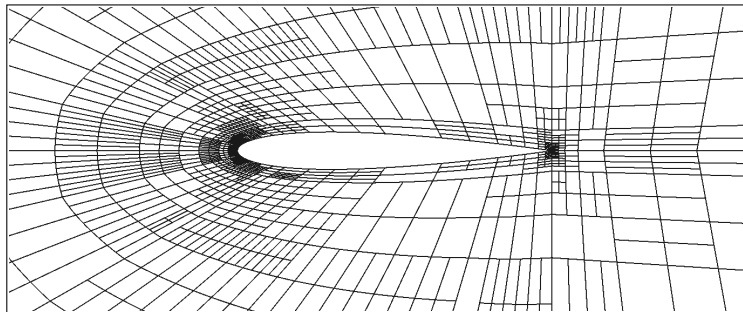
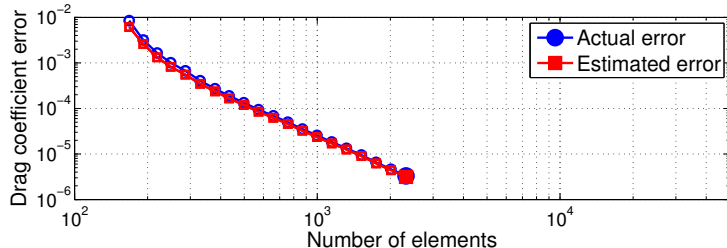


# Inviscid flow: $p = 2$ mesh sequence

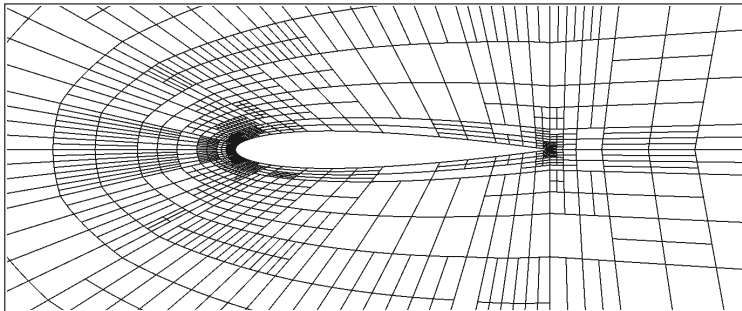
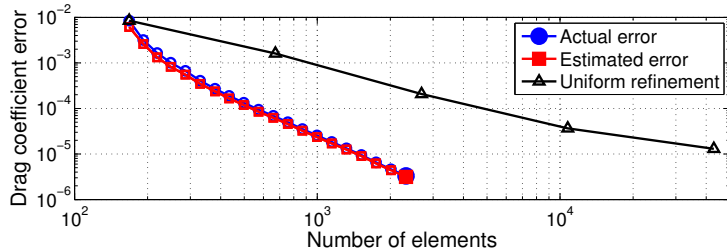




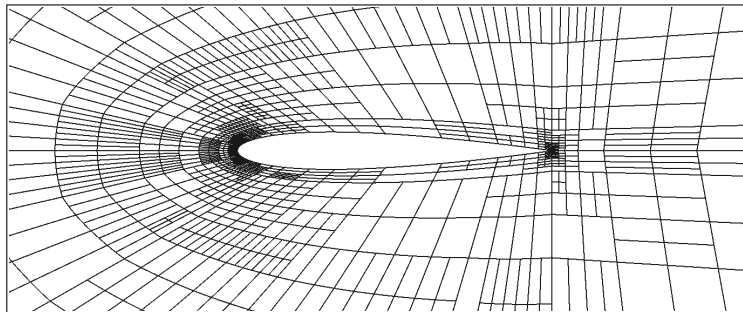
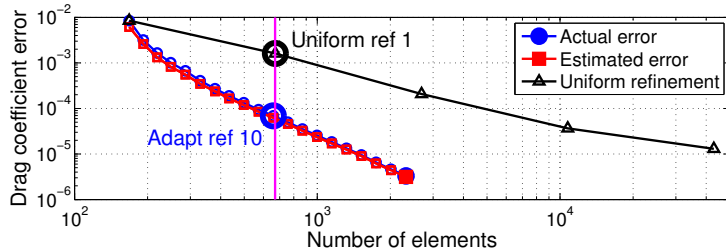
# Inviscid flow: $p = 2$ mesh sequence



# Inviscid flow: $p = 2$ mesh sequence



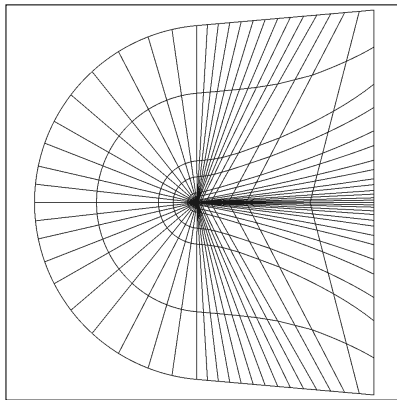
# Inviscid flow: $p = 2$ mesh sequence



# Inviscid flow: adapted/uniform comparison

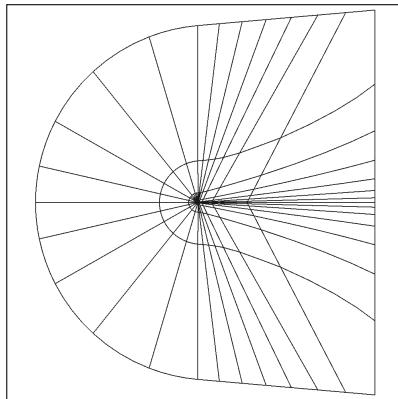
Farfield region

**Uniform refinement 1**



672 elements

**Adapt refinement 10**

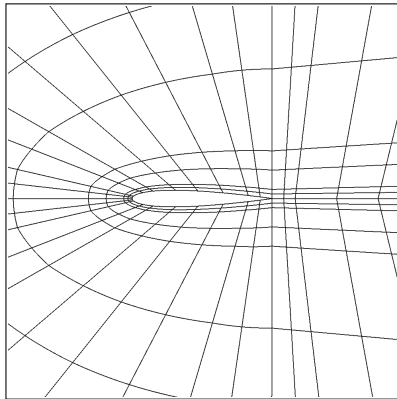


659 elements

# Inviscid flow: adapted/uniform comparison

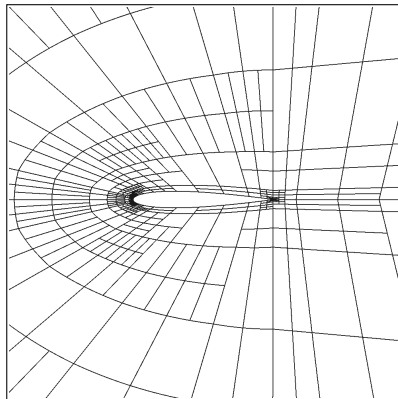
Near-field region

**Uniform refinement 1**



672 elements

**Adapt refinement 10**

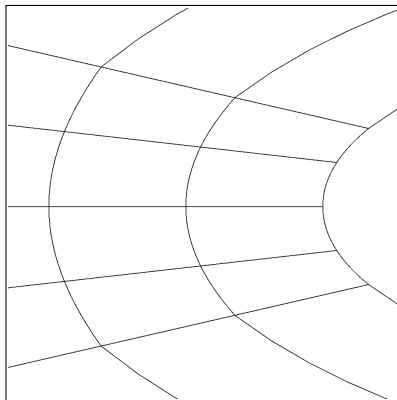


659 elements

# Inviscid flow: adapted/uniform comparison

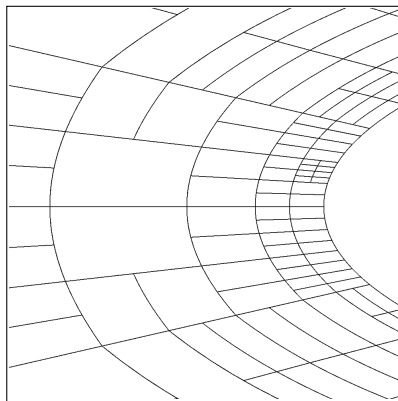
Leading edge

**Uniform refinement 1**



672 elements

**Adapt refinement 10**

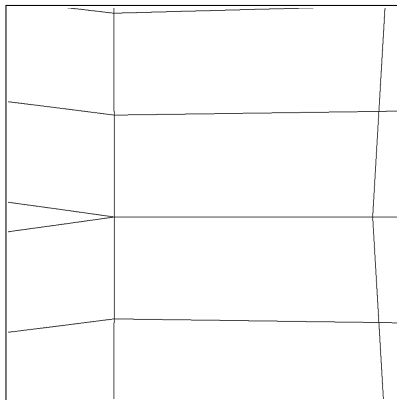


659 elements

# Inviscid flow: adapted/uniform comparison

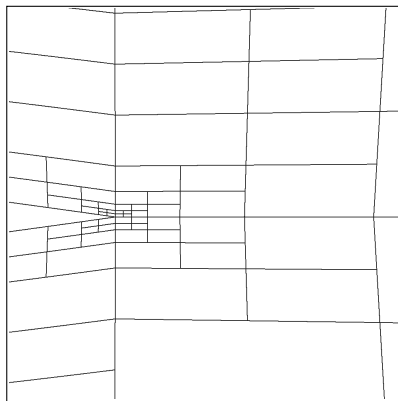
Trailing edge

**Uniform refinement 1**



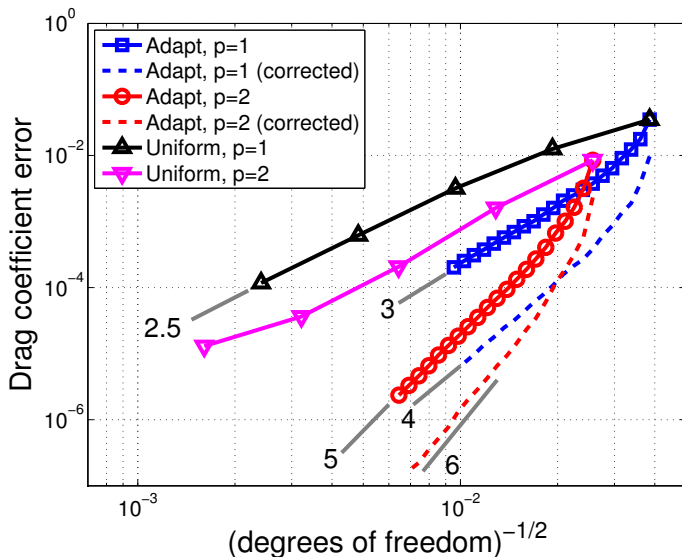
672 elements

**Adapt refinement 10**



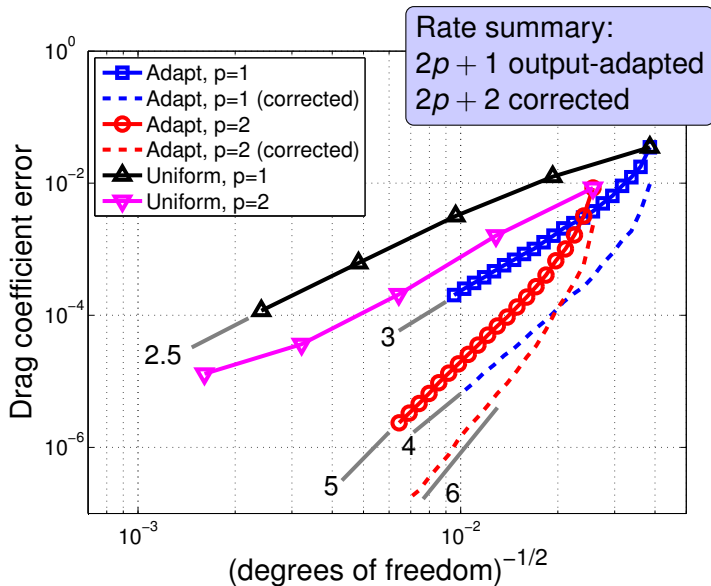
659 elements

# Inviscid flow: drag convergence (exact $\psi_h$ )

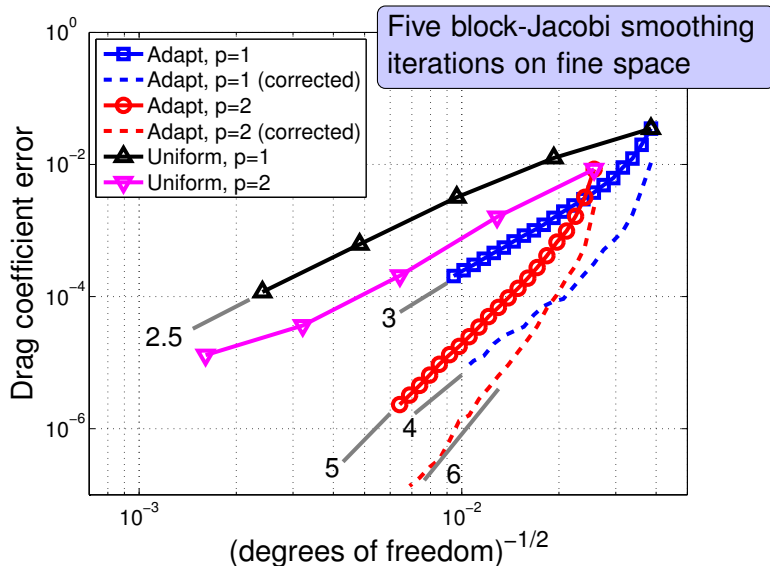




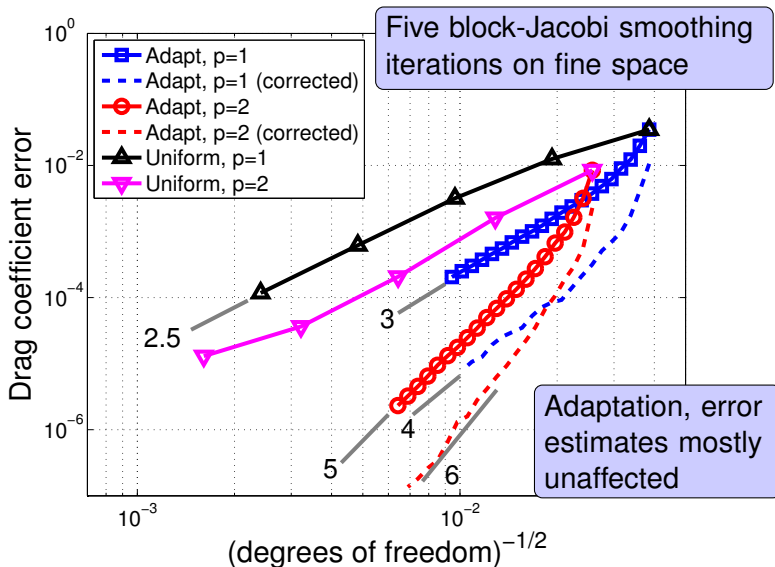
# Inviscid flow: drag convergence (exact $\psi_h$ )



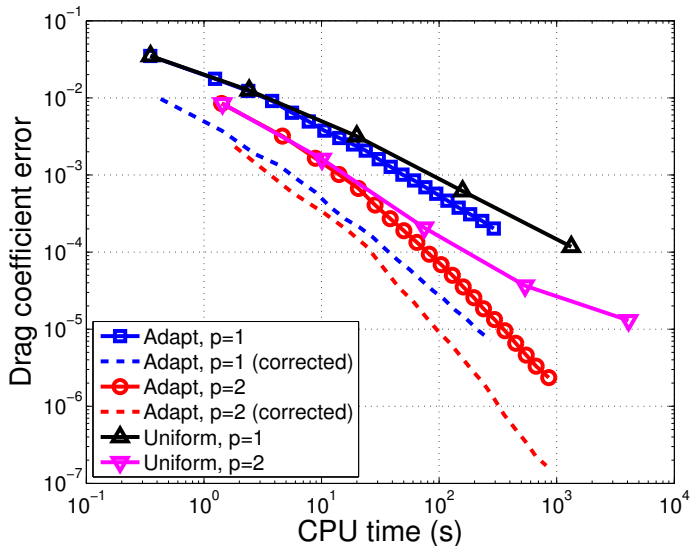
# Inviscid flow: drag convergence (approx $\psi_h$ )



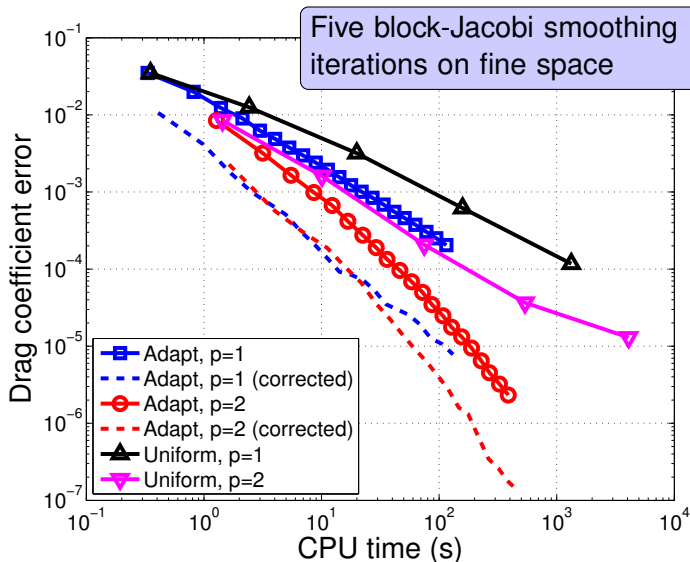
# Inviscid flow: drag convergence (approx $\psi_h$ )



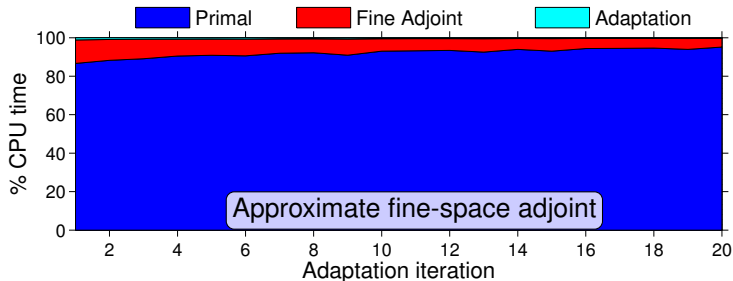
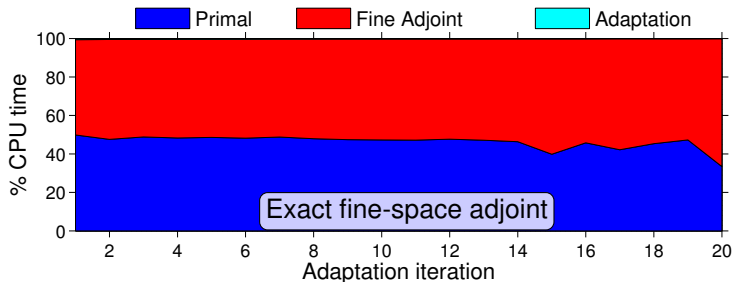
# Inviscid flow: timing comparison (exact $\psi_h$ )



# Inviscid flow: timing comparison (approx $\psi_h$ )

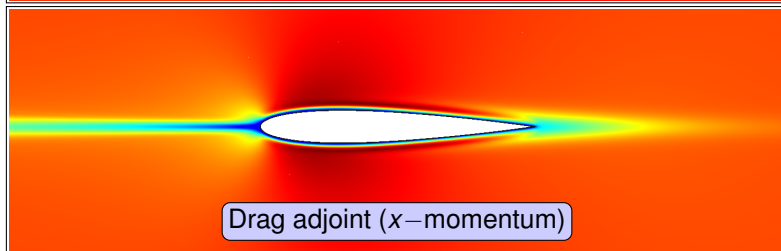
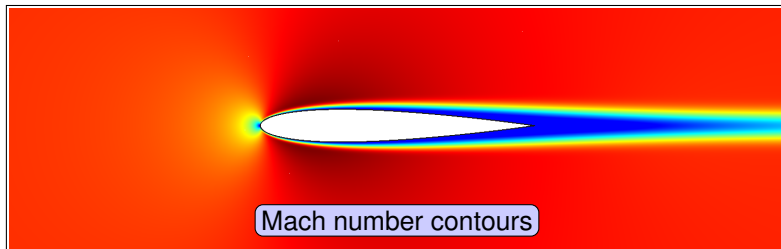


# Inviscid flow: timing percentages



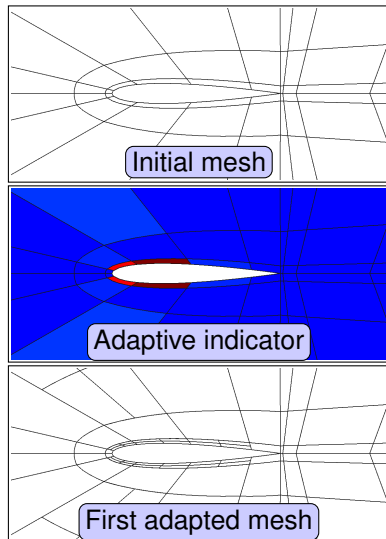
# Viscous flow over an airfoil

NACA 0012, Navier-Stokes,  $M_\infty = 0.5$ ,  $Re = 5000$ ,  $\alpha = 0^\circ$



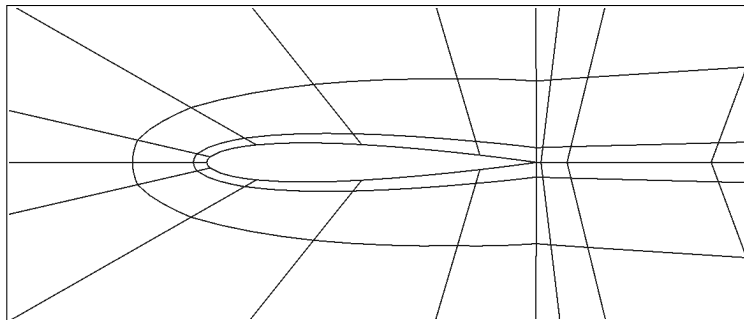
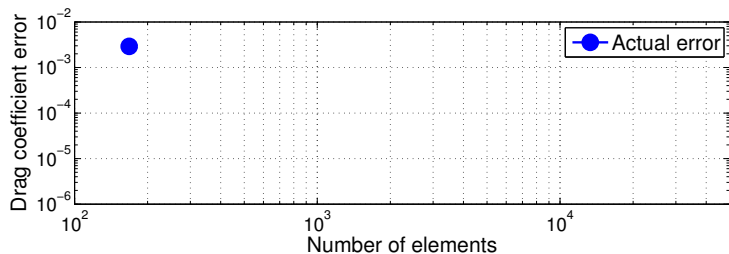
# Viscous flow: hanging-node adaptation

- Output  $J = \text{drag}$
- Isotropic hanging-node refinement
- Fine space =  $p + 1$
- Fixed fraction  $f^{\text{adapt}} = 5\%$
- 20 adaptive iterations
- No coarsening
- Use adjoint-weighted residual  $\delta J$  as correction
- “Exact” output from a  $p = 3$  fine-mesh solution
- *Right:  $p = 2$  first adaptation*

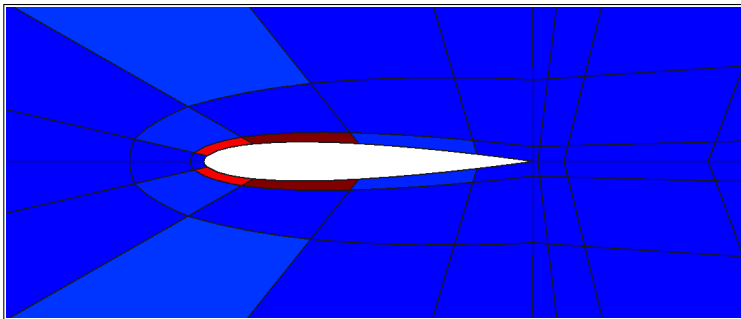
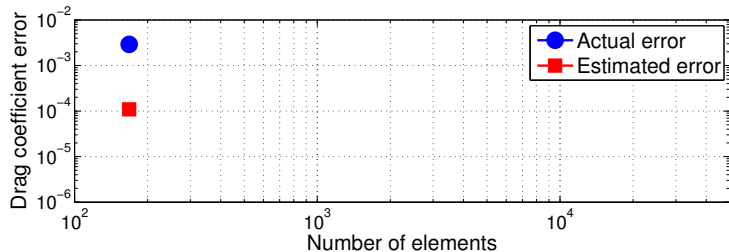




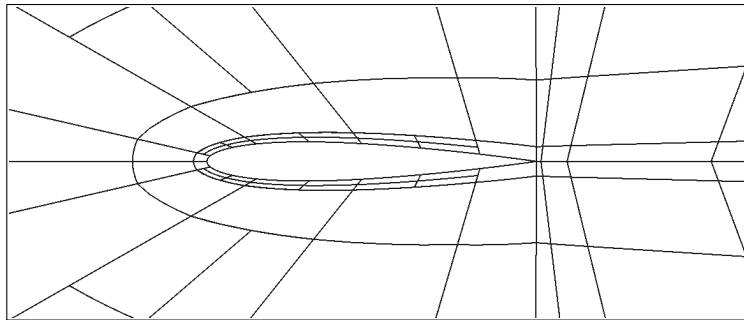
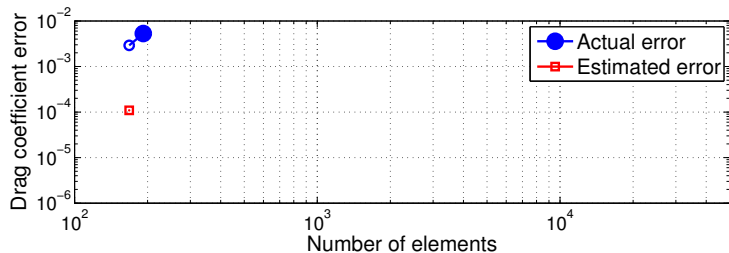
# Viscous flow: $p = 2$ mesh sequence



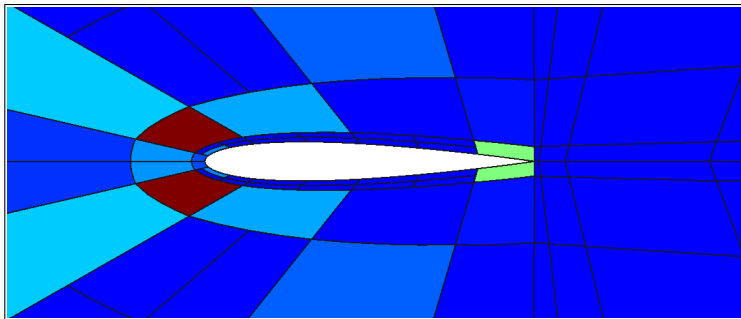
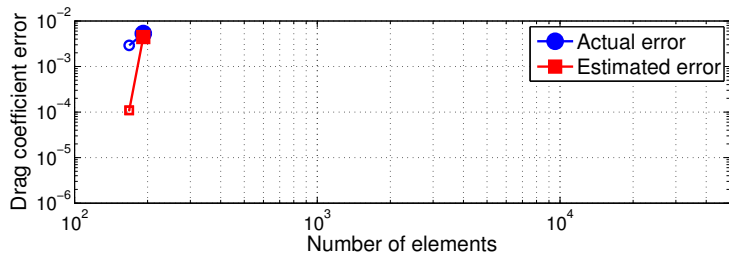
# Viscous flow: $p = 2$ mesh sequence



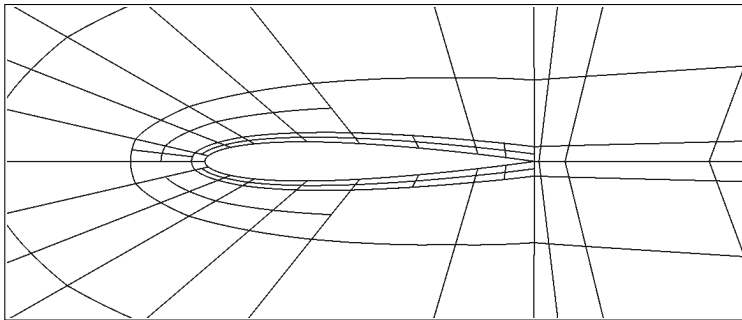
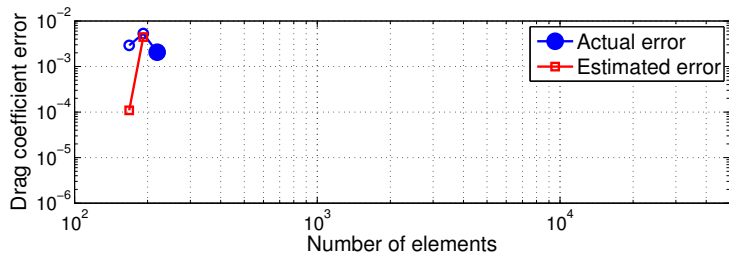
# Viscous flow: $p = 2$ mesh sequence



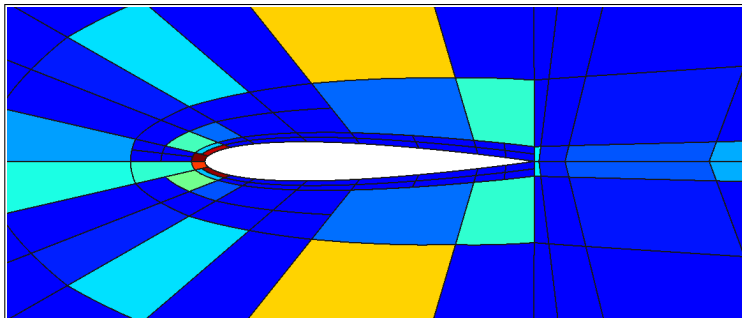
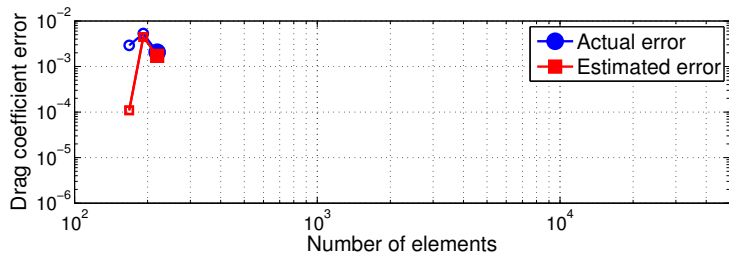
# Viscous flow: $p = 2$ mesh sequence



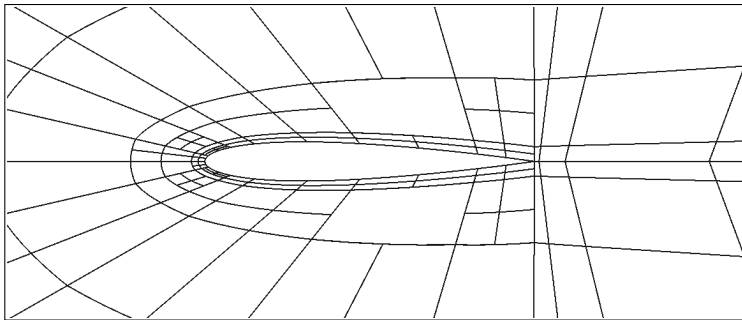
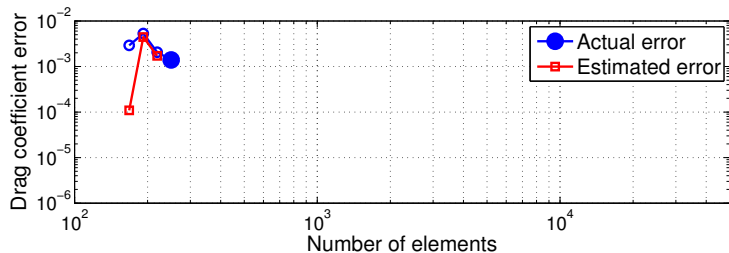
# Viscous flow: $p = 2$ mesh sequence



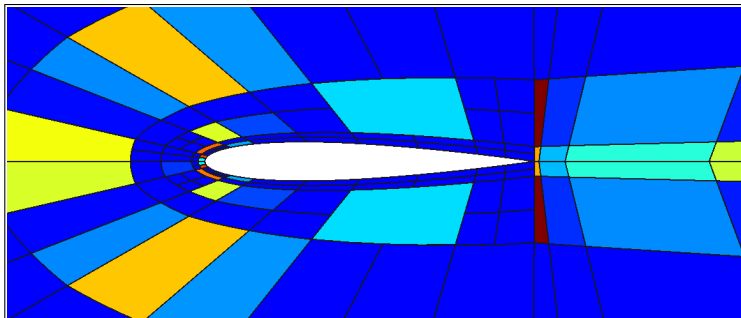
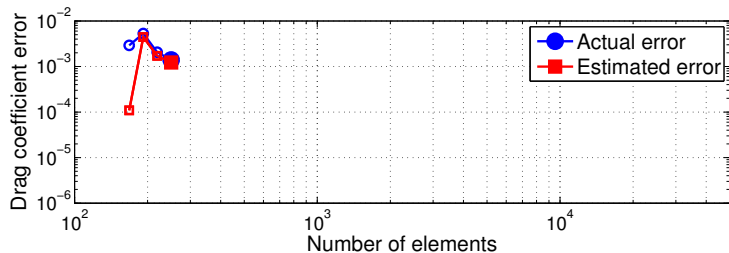
# Viscous flow: $p = 2$ mesh sequence



# Viscous flow: $p = 2$ mesh sequence

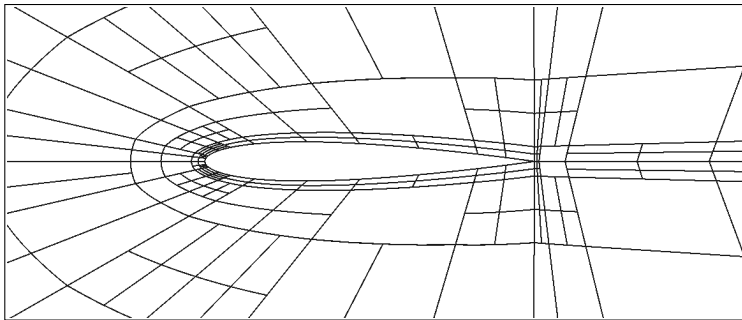
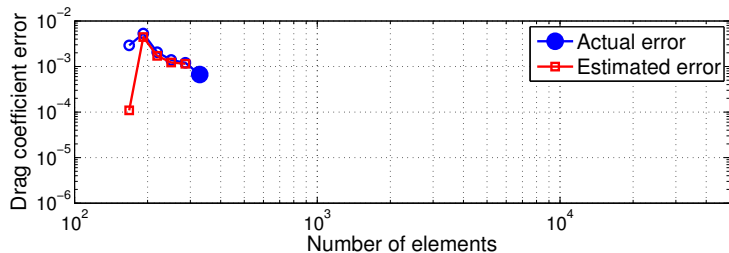


# Viscous flow: $p = 2$ mesh sequence

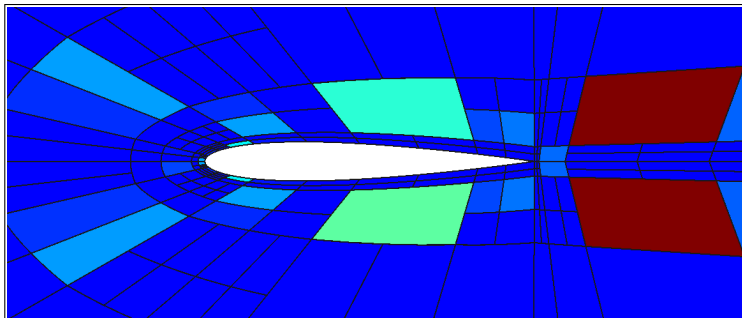
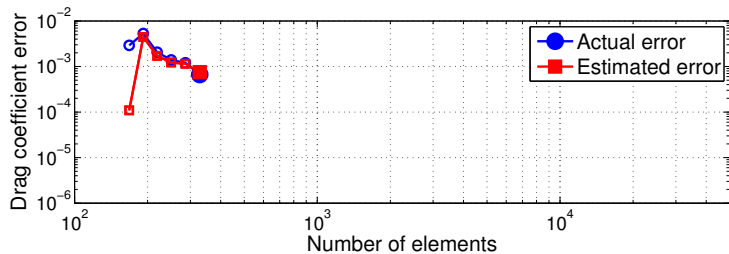




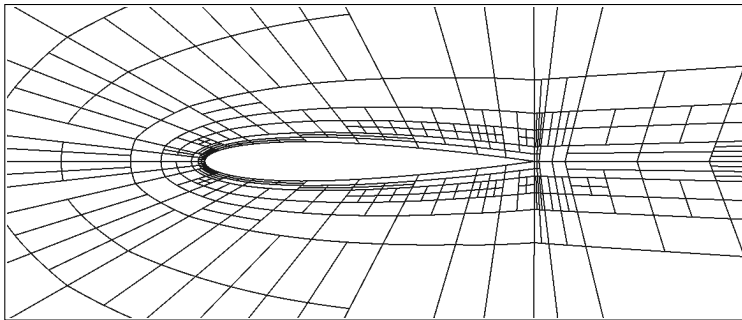
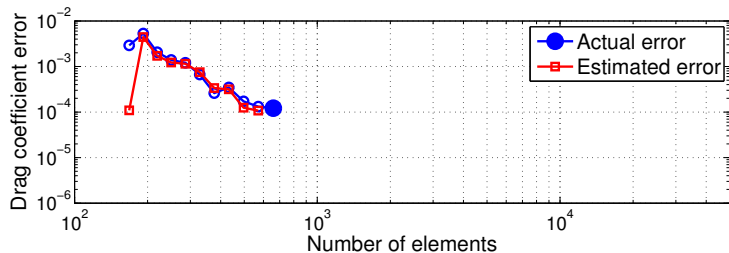
# Viscous flow: $p = 2$ mesh sequence



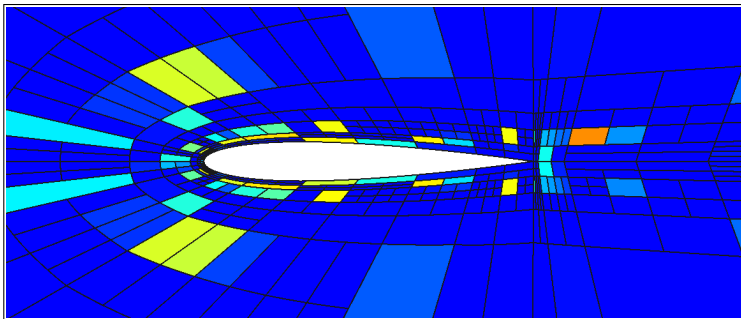
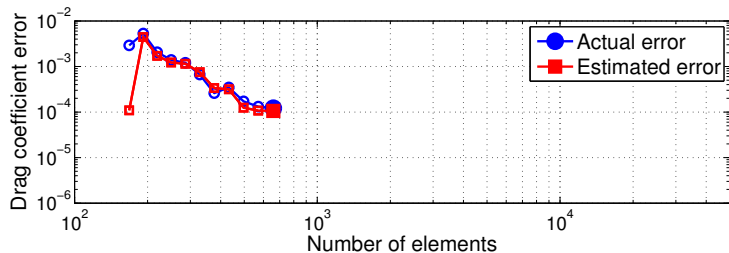
# Viscous flow: $p = 2$ mesh sequence



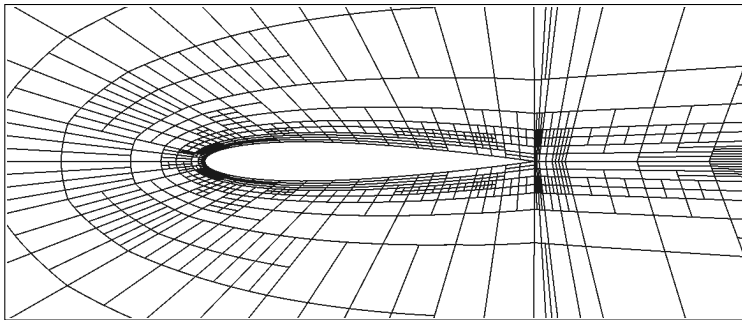
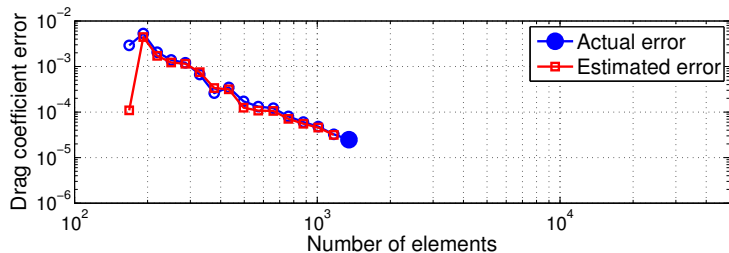
# Viscous flow: $p = 2$ mesh sequence



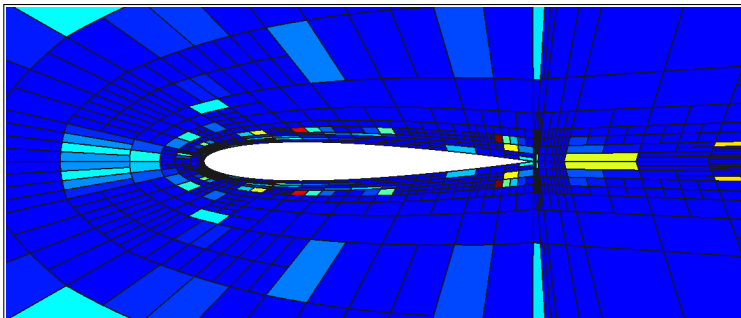
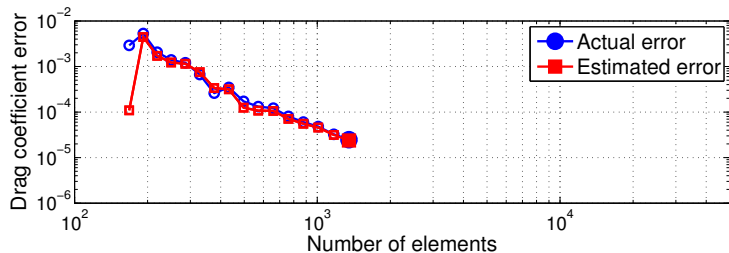
# Viscous flow: $p = 2$ mesh sequence



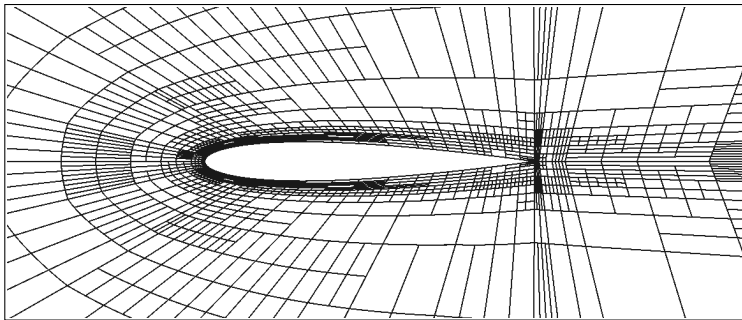
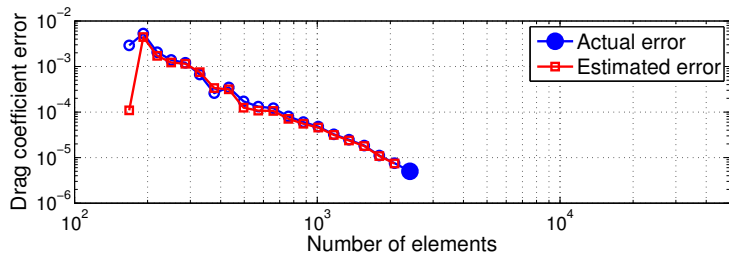
# Viscous flow: $p = 2$ mesh sequence



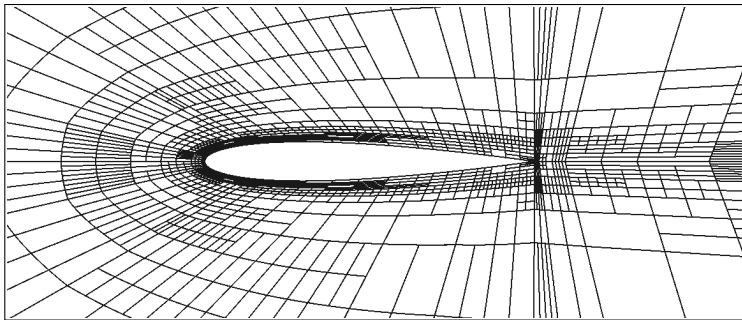
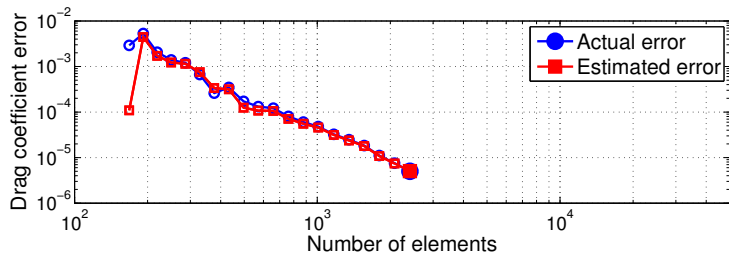
# Viscous flow: $p = 2$ mesh sequence



# Viscous flow: $p = 2$ mesh sequence

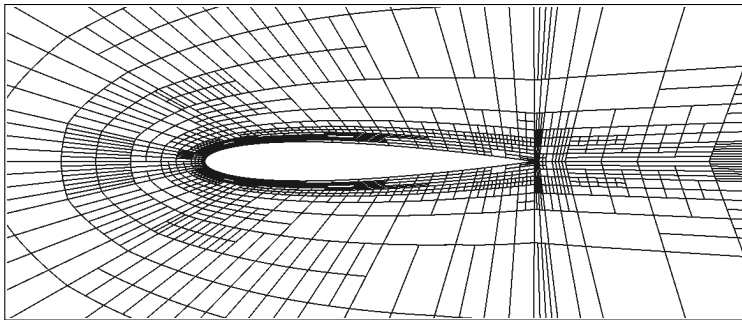
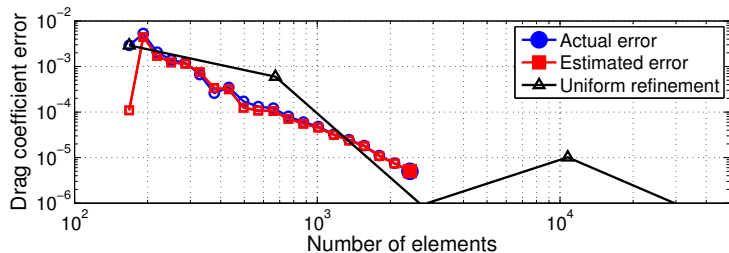


# Viscous flow: $p = 2$ mesh sequence

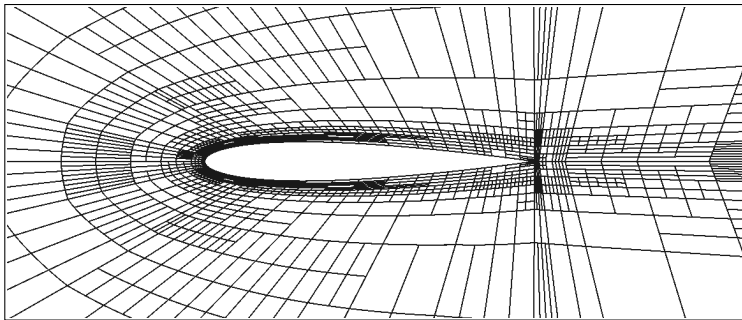
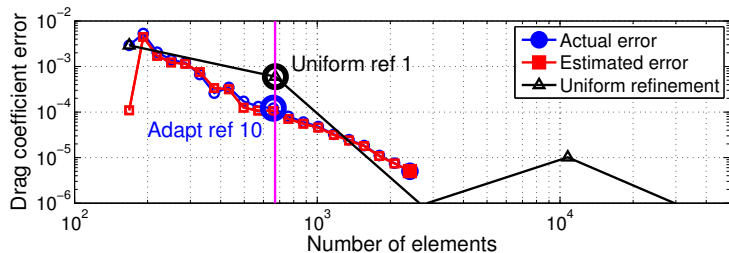




# Viscous flow: $p = 2$ mesh sequence



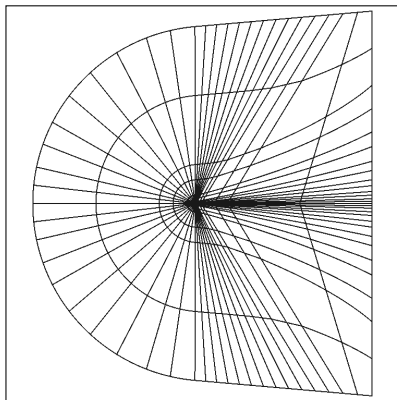
# Viscous flow: $p = 2$ mesh sequence



# Viscous flow: adapted/uniform comparison

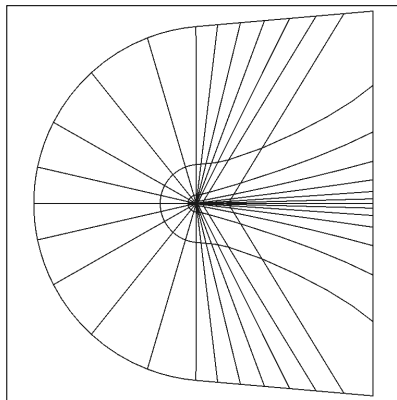
Farfield region

**Uniform refinement 1**



672 elements

**Adapt refinement 10**

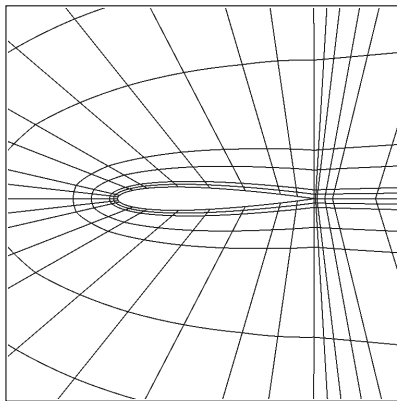


661 elements

# Viscous flow: adapted/uniform comparison

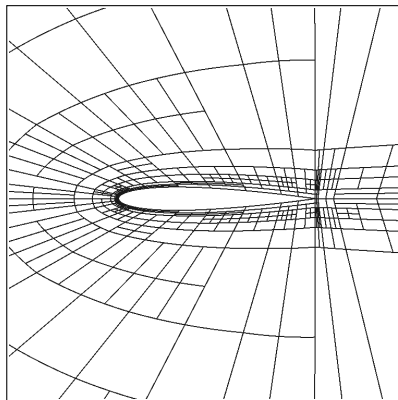
Near-field region

**Uniform refinement 1**



672 elements

**Adapt refinement 10**

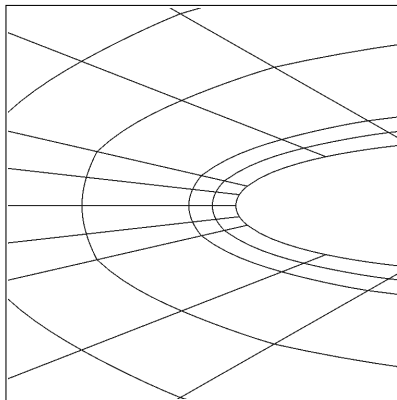


661 elements

# Viscous flow: adapted/uniform comparison

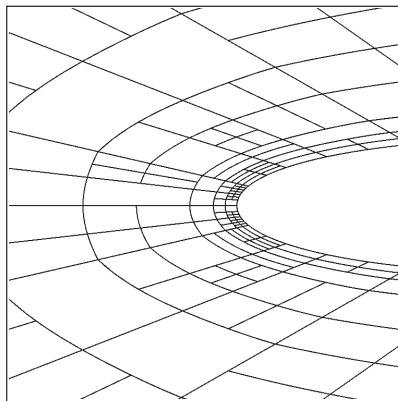
Leading edge

**Uniform refinement 1**



672 elements

**Adapt refinement 10**

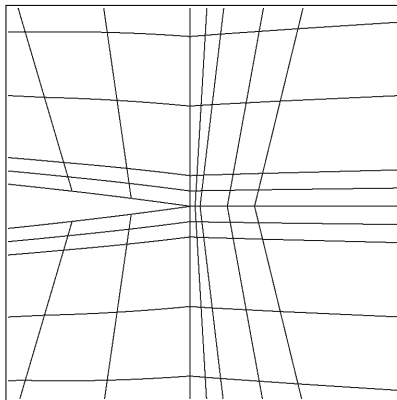


661 elements

# Viscous flow: adapted/uniform comparison

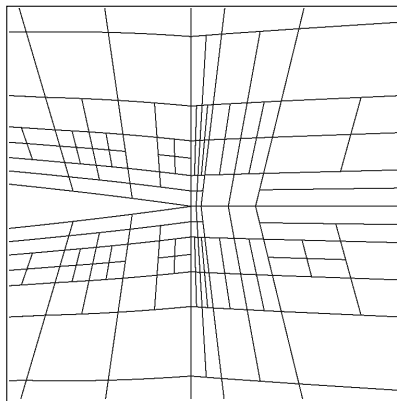
Trailing edge

**Uniform refinement 1**



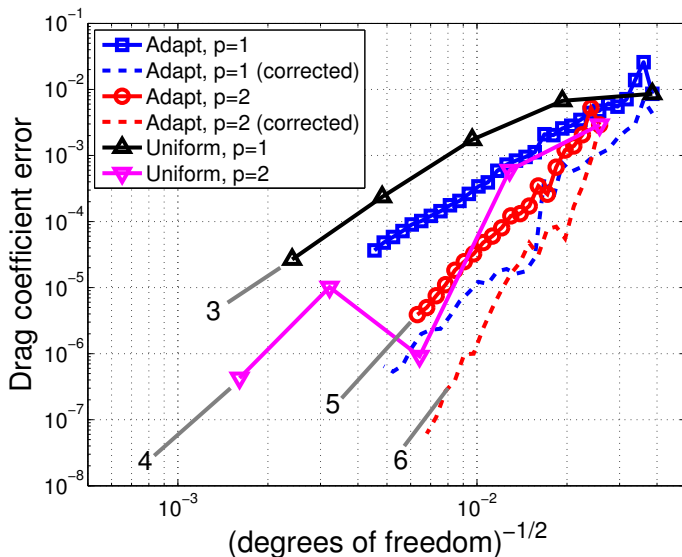
672 elements

**Adapt refinement 10**

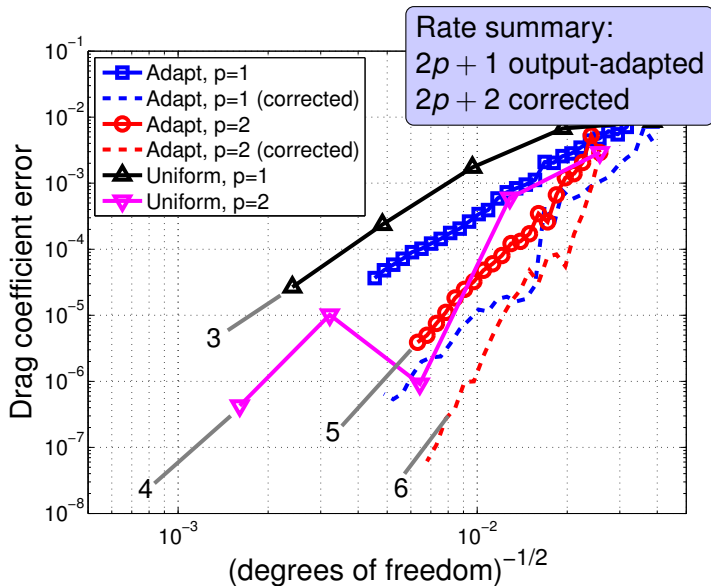


661 elements

# Viscous flow: drag convergence (exact $\psi_h$ )

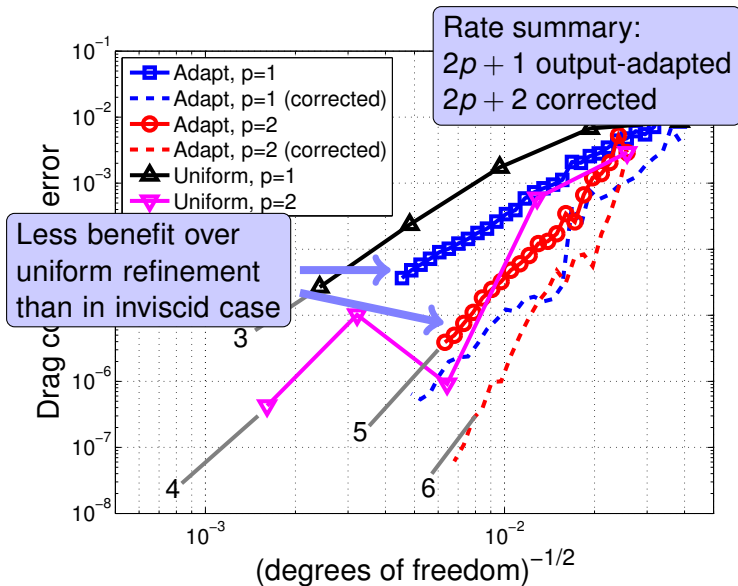


# Viscous flow: drag convergence (exact $\psi_h$ )

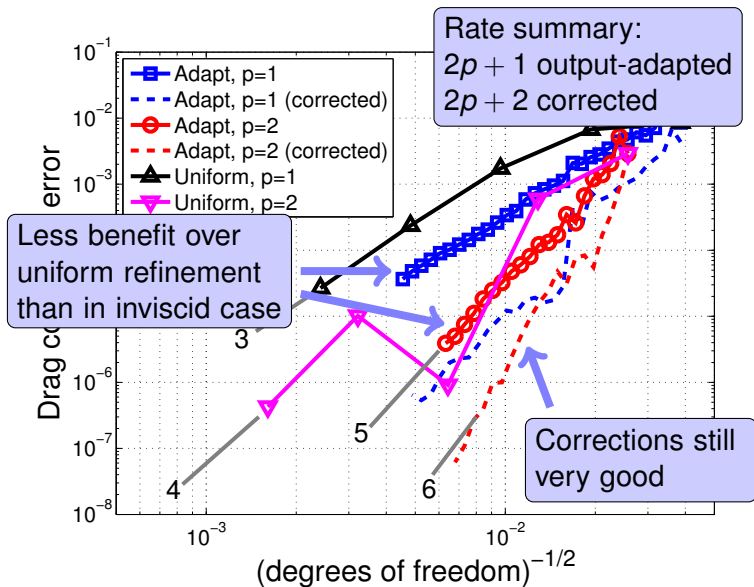




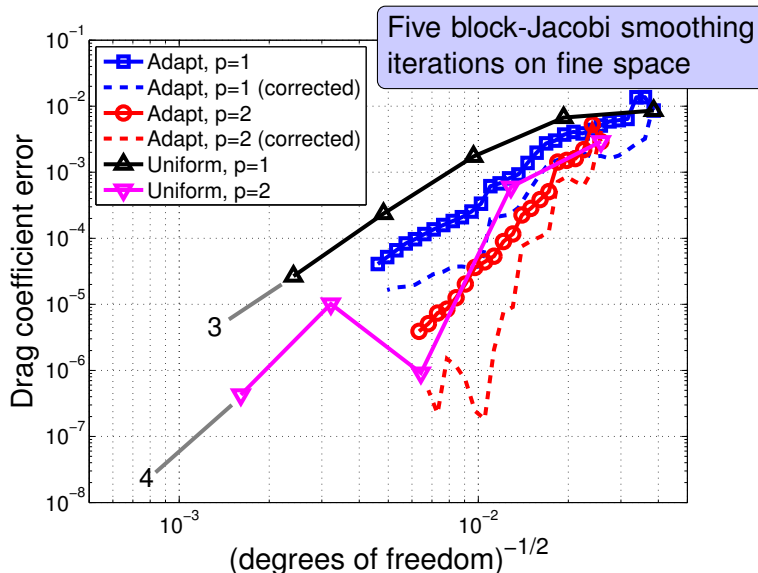
# Viscous flow: drag convergence (exact $\psi_h$ )



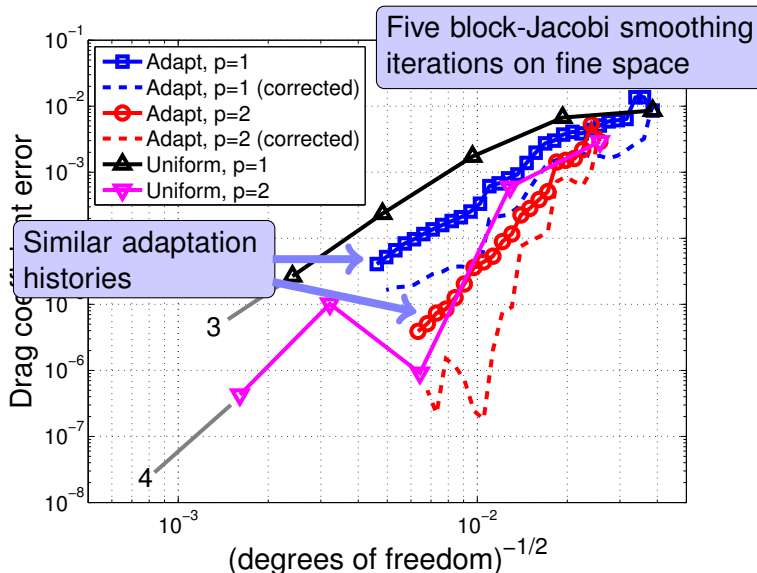
# Viscous flow: drag convergence (exact $\psi_h$ )



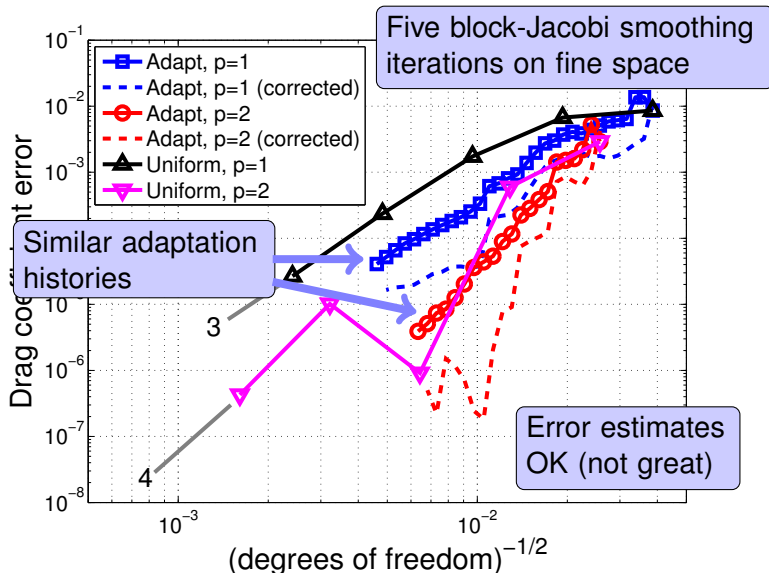
# Viscous flow: drag convergence (approx $\psi_h$ )



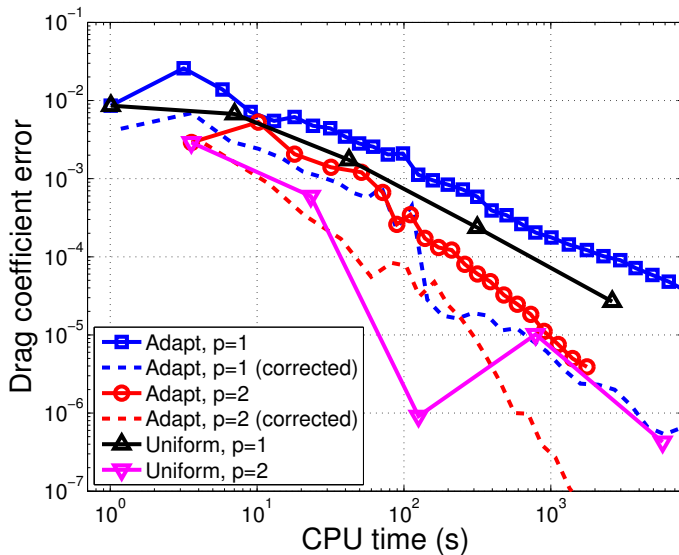
# Viscous flow: drag convergence (approx $\psi_h$ )



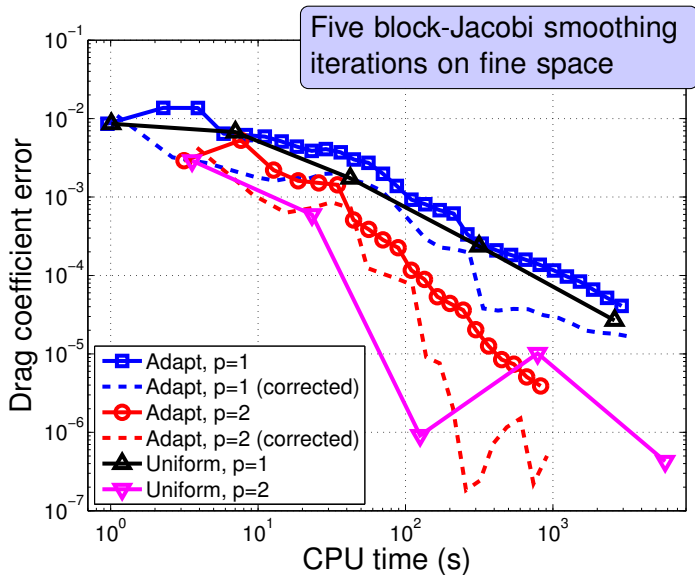
# Viscous flow: drag convergence (approx $\psi_h$ )



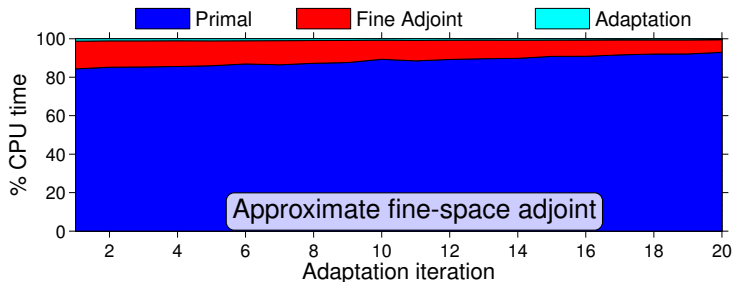
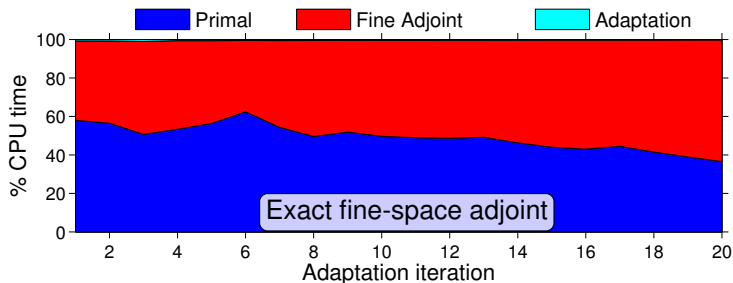
# Viscous flow: timing comparison (exact $\psi_h$ )



# Viscous flow: timing comparison (approx $\psi_h$ )



# Viscous flow: timing percentages





# Summary

- We can quantify numerical error and adapt the mesh to reduce it at its source
- This requires an adjoint and fine-space calculations
- Exact fine-space adjoints yield effective error estimates that we can use as corrections
- Approximate fine-space adjoints (e.g. via Jacobi smoothing) are still good for adaptation
- So far we only looked at:
  - Greedy hanging-node adaptation: limited in mechanics and strategy
  - Isotropic refinement: not efficient for viscous problems at high Reynolds numbers
  - Steady problems: unsteady simulations add another dimension and complicate error estimation